

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC THÀNH ĐÔNG



KHÓA LUẬN TỐT NGHIỆP
TÌM HIỂU VÀ PHÁT TRIỂN GAME HYPER-CASUAL
“SHOOTING ZOMBIE” TRÊN UNITY

Giảng viên HD:	ThS. Hoàng Anh Tuấn
Sinh viên:	Khuong Hoài Nam
Lớp:	1A11_CNTTK11
Ngành:	Công nghệ thông tin

Hải Phòng, Năm 2025

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn sâu sắc tới ThS. Hoàng Anh Tuấn, giảng viên hướng dẫn, người đã tận tâm hỗ trợ, định hướng và truyền đạt những kiến thức chuyên môn trong suốt quá trình học tập và thực hiện đề tài. Những định hướng và góp ý của thầy đã giúp em hoàn thiện tư duy, phương pháp làm việc và phát triển kỹ năng cần thiết trong lĩnh vực phát triển game.

Em cũng xin trân trọng cảm ơn các thầy, cô trong Khoa Công nghệ Thông tin, Trường Đại học Thành Đông đã tạo điều kiện thuận lợi và cung cấp môi trường học tập tích cực trong suốt thời gian học tập tại trường. Những kiến thức và kỹ năng được trang bị là nền tảng quan trọng giúp em có thể triển khai và hoàn thành khóa luận này.

Em xin chân thành cảm ơn!

Hải Phòng, Năm 2025

Sinh viên thực hiện

Khương Hoài Nam

CAM ĐOAN

Em tên là: Khương Hoài Nam

Mã số sinh viên: 2011100008

Em xin cam đoan rằng đề tài “[TÌM HIỂU VÀ PHÁT TRIỂN GAME HYPER-CASUAL “SHOOTING ZOMBIE” TRÊN UNITY]” là kết quả nghiên cứu do chính em thực hiện.

Toàn bộ nội dung, số liệu, hình ảnh và tài liệu tham khảo được sử dụng trong khóa luận đều được thu thập một cách trung thực, có nguồn gốc rõ ràng và đã được trích dẫn đầy đủ theo đúng quy định. Em tuyệt đối không sao chép, không sử dụng nội dung từ bất kỳ khóa luận, tài liệu hay công trình nghiên cứu nào khác mà không ghi rõ nguồn.

Em xin chịu hoàn toàn trách nhiệm trước nhà trường về tính trung thực và cam kết của mình đối với khóa luận này.

Hải Phòng, Năm 2025

Sinh viên thực hiện

Khương Hoài Nam

MỤC LỤC

LỜI CẢM ƠN	I
CAM ĐOAN	II
MỤC LỤC	III
DANH MỤC SƠ ĐỒ HÌNH ẢNH	VII
DANH MỤC THUẬT NGỮ	IX
CHƯƠNG 1: MỞ ĐẦU	1
1.1. Giới thiệu đề tài nghiên cứu	1
1.2. Lý do chọn đề tài	1
1.3. Đối tượng nghiên cứu	2
1.4. Mục đích nghiên cứu	2
1.5. Nội dung chính của báo cáo	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	6
2.1. Giới thiệu Unity Engine	6
2.1.1. Tổng quan về Unity Engine	6
2.1.2. Lịch sử và tầm ảnh hưởng	6
2.1.3. Các tính năng cốt lõi và nổi bật	7
2.1.4. Ứng dụng Unity trong mô phỏng, kiến trúc và công nghiệp	9
2.1.5. Cộng đồng Unity tại Việt Nam và trên thế giới	10
2.1.6. Kết luận	10
2.2. Giới thiệu ngôn ngữ lập trình C#	10
2.3. Giới thiệu thể loại game hyper-casual	12
2.3.1. Giới thiệu chung	12
2.3.2. Các đặc điểm cốt lõi	12
2.3.3. Mô hình kinh doanh và kiếm tiền	13

2.3.4. Ví dụ tiêu biểu	14
2.3.5. Kết luận	15
CHƯƠNG 3: CÀI ĐẶT MÔI TRƯỜNG	16
3.1. Giới thiệu môi trường làm việc	16
3.1.1. Mục tiêu	16
3.1.2. Hướng dẫn cài đặt Unity	16
3.1.3. Tạo Project mới trong Unity	18
3.1.4. Môi trường làm việc trong Unity	19
3.1.6. Một số thao tác cơ bản	22
3.1.7. Kết luận	23
CHƯƠNG 4: GIỚI THIỆU SẢN PHẨM	24
4.1. Giới thiệu chung	24
4.1.1. Tên khóa luận	24
4.1.2. Ý tưởng và nội dung trò chơi	24
4.1.3. Các chế độ chơi (Mode)	24
4.1.4. Tính năng nổi bật	27
4.1.5. Công nghệ và công cụ sử dụng	27
4.1.6. Thiết kế giao diện	28
4.1.7. Mục tiêu phát triển	30
4.3. Công nghệ và công cụ sử dụng	31
4.3.1. Unity Engine	31
4.3.2. Ngôn ngữ lập trình C#	32
4.3.3. Spine Animation	32
4.3.4. Hệ thống cửa hàng– IAP (In-App Purchasing)	33
4.4. Gameplay và giao diện người dùng (UI/UX)	34
4.4.1. Thiết kế Gameplay	34

4.4.2. Thiết kế giao diện người dùng (UI/UX)	37
4.4.3. Nguyên tắc thiết kế UX được áp dụng	40
4.5. Hệ thống nhiệm vụ và phần thưởng	41
4.5.3. Hệ thống phần thưởng	42
4.5.4. Động lực hóa người chơi	43
4.6. Hệ thống điều khiển và Enemy(AI)	44
4.6.1. Hệ thống điều khiển nhân vật người chơi	44
4.6.2. Hệ thống AI của zombie	44
4.6.3. Tối ưu hiệu suất AI	45
4.6.4. Tính năng đặc biệt	45
4.7. Hệ thống nâng cấp	45
4.7.1. Mục tiêu của hệ thống	45
4.7.2. Các loại nâng cấp trong game	45
4.7.3. Giao diện nâng cấp (Upgrade UI)	47
4.7.4. Hệ thống cửa hàng (Shop)	47
4.7.5. Cơ chế kích thích người chơi nâng cấp	48
4.8. Quản lý dữ liệu người chơi và lưu trữ	48
4.8.1. Mục tiêu của hệ thống lưu trữ dữ liệu	48
4.8.2. Phương pháp lưu dữ liệu sử dụng trong game	49
4.8.3. Cơ chế tự động lưu / load dữ liệu	50
CHƯƠNG 5: KIỂM THỬ VÀ TỐI ƯU HIỆU NĂNG	52
5.1. Mục tiêu kiểm thử	52
5.1.2. Các hình thức kiểm thử được áp dụng	52
5.1.3. Tối ưu hiệu suất	53
5.1.4. Kết quả kiểm thử	53
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	54

1. Kết luận	54
2. Hướng phát triển trong tương lai	54
3. Đánh giá cá nhân	55
Tài liệu tham khảo	56

DANH MỤC SƠ ĐỒ HÌNH ẢNH

Hình 2.1. Giao diện trang chủ Unity	6
Hình 2.2. Unity Asset Store	9
Hình 2.3. Vòng lặp game	13
Hình 2.4. Game Helix Jump trên Google Play	14
Hình 2.5. Game Paper.io trên Google Play	14
Hình 2.6. Game Bridge Race io trên Google Play	15
Hình 2.7. Game Tall Man Run trên Google Play	15
Hình 3.1. Giao diện trang tải về Unity Hub.	16
Hình 3.2. Giao diện Installs – chọn phiên bản Unity để cài.	17
Hình 3.3. Giao diện tạo Project mới.	19
Hình 3.4. Cửa sổ Scene	19
Hình 3.5. Cửa sổ Game	20
Hình 3.6. Cửa sổ Hierarchy	20
Hình 3.7. Cửa sổ Project	21
Hình 3.8. Cửa sổ Inspector	21
Hình 3.9. Các nút điều khiển chính	22
Hình 4.1. Chọn mode story (Campaign Mode)	24
Hình 4.2. Game Play Mode Story	24
Hình 4.3. Chọn Mode Endless	25
Hình 4.4. Game Play Mode Endless	25
Hình 4.5. Chọn chơi Mode Boss	25
Hình 4.6. Chọn Boss chiến đấu trong mode boss	26
Hình 4.7. Game Play Mode Boss	26
Hình 4.8. Chọn Mode Collection	26
Hình 4.9. Game play mode Collection	27
Hình 4.10. Màn hình chính	28
Hình 4.11. Màn hình chọn mode chơi	28
Hình 4.12. Màn hình Game Play	29
Hình 4.13. Màn hình chiến thắng	29
Hình 4.14. Màn hình đếm ngược thất bại	30

Hình 4.15. Màn hình thất bại	30
Hình 4.16. Giao diện Unity	32
Hình 4.17. Một phần nhỏ mã C# được viết trong dự án	32
Hình 4.18. Nhân vật được sử dụng Spine Animation	33
Hình 4.19. Giao diện điều khiển trong game	34
Hình 4.20. Dữ liệu các Enemy trong game	35
Hình 4.21. Dữ liệu chỉ số súng trong Game	36
Hình 4.22. Dữ liệu súng theo level trong game	36
Hình 4.23. Ví dụ dữ liệu minh họa trong game	37
Hình 4.24. Màn hình bối cảnh chính	38
Hình 4.25. Màn hình chọn chế độ	38
Hình 4.26. Màn hình Game Play	39
Hình 4.27. Hệ thống nâng cấp khi qua wave mới	39
Hình 4.28. Màn hình chiến thắng game	39
Hình 4.29. Hình đếm ngược khi thua	40
Hình 4.30. Màn hình thua game	40
Hình 4.31. Popup hệ thống nhiệm vụ	41
Hình 4.32. Popup nhận thưởng	42
Hình 4.33. Popup phần thưởng mỗi trận	42
Hình 4.34. Popup Lucky Spin	43
Hình 4.35. Popup nhận quà hằng ngày	43
Hình 4.36. Màn hình nâng cấp xe	46
Hình 4.37. Màn hình nâng cấp nhân vật trên xe	46
Hình 4.38. Màn hình nâng cấp đạn	47
Hình 4.39. Hình ảnh hệ thống shop	48
Hình 4.40. Hệ thống ScriptableObject trong game	50
Hình 4.41. Hệ thống lưu game	51

DANH MỤC THUẬT NGỮ

Thuật Ngữ	Giải Thích
Game Engine	Bộ khung phần mềm phát triển game, ví dụ: Unity, Unreal Engine.
Hyper-Casual Game	Thể loại game đơn giản, dễ chơi, hướng đến số đông.
Unity	Một trong những Game Engine phổ biến nhất, hỗ trợ đa nền tảng.
Rendering Engine	Thành phần xử lý đồ họa, ánh sáng, bóng đổ trong game.
Physics Engine	Thành phần mô phỏng vật lý: trọng lực, va chạm, chuyển động.
Audio Engine	Quản lý âm thanh: nhạc nền, hiệu ứng âm thanh, giọng nói.
Scripting	Lập trình logic game, ví dụ: C#, C++, GDScript.
AI (Artificial Intelligence)	Trí tuệ nhân tạo giúp điều khiển NPC (nhân vật không điều khiển).
Animation	Diễn hoạt nhân vật và đối tượng.
Networking	Kết nối mạng trong game nhiều người chơi.
Component-Based Architecture	Kiến trúc dựa trên thành phần trong Unity.
Prefab	Đối tượng mẫu tái sử dụng nhiều lần trong game.
Scene	Một màn chơi hoặc môi trường game.
MonoBehaviour	Lớp cơ bản trong Unity cho phép đối tượng tham gia vào vòng đời game.
Asset Store	Kho tài nguyên của Unity gồm mô hình, âm thanh, code mẫu.
VR (Virtual Reality)	Thực tế ảo, ứng dụng công nghệ VR trong

	game.
AR (Augmented Reality)	Thực tế tăng cường, kết hợp hình ảnh thực tế với đối tượng ảo.
Cloud Gaming	Chơi game qua nền tảng điện toán đám mây.
Procedural Content Generation	Tạo nội dung tự động bằng thuật toán (bản đồ, level).
Design Pattern	Mẫu thiết kế phần mềm (Singleton, Observer, State, Object Pooling...).
Singleton Pattern	Đảm bảo chỉ có một đối tượng duy nhất tồn tại xuyên suốt chương trình.
Observer Pattern	Mẫu thiết kế sự kiện, theo dõi thay đổi trạng thái.
OOP (Object-Oriented Programming)	Lập trình hướng đối tượng.
Encapsulation	Đóng gói thông tin.
Inheritance	Kế thừa lớp cha.
Polymorphism	Đa hình, nhiều cách thực thi hành vi.
Abstraction	Trừu tượng hóa, ẩn chi tiết phức tạp.
Git	Công cụ quản lý phiên bản mã nguồn.
GitLab/GitHub/Bitbucket	Nền tảng lưu trữ mã nguồn online.
CI/CD	Tích hợp và triển khai liên tục.
FPS (Frames Per Second)	Số khung hình hiển thị mỗi giây.
UI/UX	Giao diện và trải nghiệm người dùng.
Universal Render Pipeline (URP)	Hệ thống render nhẹ của Unity cho mobile/VR.
High Definition Render Pipeline (HDRP)	Hệ thống render chất lượng cao trong Unity.

CHƯƠNG 1: MỞ ĐẦU

1.1. Giới thiệu đề tài nghiên cứu

Trong bối cảnh công nghệ di động phát triển mạnh mẽ, ngành công nghiệp game đang trở thành một trong những lĩnh vực giải trí có tốc độ tăng trưởng cao nhất hiện nay. Sự phổ biến của điện thoại thông minh và nhu cầu giải trí ngắn hạn đã thúc đẩy sự bùng nổ của thể loại game hyper-casual – dòng game đơn giản, dễ tiếp cận và thu hút đông đảo người chơi trên toàn thế giới.

Bên cạnh đó, Unity – một trong những nền tảng phát triển game đa nền tảng phổ biến nhất – đã mở ra cơ hội lớn cho những lập trình viên trẻ, sinh viên và các nhóm phát triển độc lập trong việc hiện thực hóa ý tưởng game với chi phí thấp và hiệu suất cao.

Xuất phát từ niềm đam mê với lập trình game nói chung và mong muốn tiếp cận thực tế quá trình thiết kế – phát triển một sản phẩm game hoàn chỉnh, em đã lựa chọn đề tài: "Tìm hiểu và phát triển game Hyper-Casual: Shooting Zombie trên Unity" làm đồ án tốt nghiệp.

Đề tài tập trung nghiên cứu quy trình xây dựng một trò chơi đơn giản nhưng hấp dẫn, tối ưu hóa cho nền tảng di động. Sản phẩm hướng tới việc vận dụng hiệu quả các kiến thức đã học về lập trình hướng đối tượng, cấu trúc dữ liệu, giao diện người dùng, xử lý tương tác người chơi, và quản lý dữ liệu trong môi trường phát triển Unity.

Thông qua đề tài này, em không chỉ mong muốn tạo ra một sản phẩm thực tiễn có thể phát hành thử nghiệm, mà còn góp phần tích lũy kinh nghiệm phát triển game và chia sẻ tài liệu tham khảo cho cộng đồng sinh viên cùng lĩnh vực.

1.2. Lý do chọn đề tài

Trong bối cảnh công nghệ di động phát triển mạnh mẽ, sản xuất game đã trở thành một phần quan trọng trong ngành giải trí. Các thiết bị di động hiện đại không chỉ cung cấp hiệu suất tốt hơn mà còn mang đến trải nghiệm đồ họa phong phú, thu hút người chơi.

Lập trình game không chỉ là một ngành nghề có tiềm năng sinh lợi cao mà còn là lĩnh vực công nghệ cao đòi hỏi sự sáng tạo và tư duy. Game mang lại niềm

vui và sự thoải mái cho người chơi, với nhiều thể loại khác nhau như game 2D, 3D, game giáo dục, game hành động, và đặc biệt là game hyper-casual.

Với sự xuất hiện của nhiều công cụ phát triển game, Unity nổi bật nhờ khả năng xuất ra đa nền tảng và mạnh mẽ trong phát triển game 2D cũng như 3D, thực tế ảo và nhiều ứng dụng khác. Nhận thức được tầm quan trọng và tiềm năng của game hyper-casual, em đã quyết định chọn đề tài “Tìm hiểu và phát triển Game thể loại Hyper-Casual trên Unity” làm đồ án tốt nghiệp của mình.

1.3. Đối tượng nghiên cứu

Phân nghiên cứu tập trung vào việc tìm hiểu và sử dụng công cụ Unity trong môi trường phát triển game 2D, nhằm nắm vững cách xây dựng và quản lý các đối tượng trong không gian hai chiều. Bên cạnh đó, đề tài cũng đi sâu tìm hiểu các trò chơi thuộc thể loại hyper-casual để phân tích lối chơi đơn giản nhưng hấp dẫn, từ đó rút ra kinh nghiệm xây dựng sản phẩm. Ngoài ra, việc nghiên cứu hệ điều hành Android và iOS giúp hiểu rõ hơn về cách thức triển khai và tối ưu trò chơi trên các nền tảng di động phổ biến. Thuật toán mô phỏng tương tác vật lý cũng được nghiên cứu nhằm tạo ra các hiệu ứng chuyển động và va chạm chân thực, góp phần nâng cao trải nghiệm người chơi. Cuối cùng, đề tài phân tích lý thuyết về tương tác người chơi để xây dựng cơ chế điều khiển và giao diện phù hợp, mang đến sự thuận tiện và hứng thú cho người dùng.

1.4. Mục đích nghiên cứu

Mục đích chính của đề tài là xây dựng một sản phẩm game Hyper-Casual hoàn chỉnh mang tên “Shooting Zombie”, qua đó hiện thực hóa toàn bộ quy trình phát triển một trò chơi từ ý tưởng ban đầu đến giai đoạn có thể phát hành thử nghiệm.

Trước hết, đề tài tập trung vào việc phát triển một trò chơi có lối chơi đơn giản nhưng hấp dẫn, phù hợp với đặc trưng của thể loại Hyper-Casual. Người chơi chỉ cần thực hiện thao tác chạm hoặc kéo để điều khiển súng, qua đó nhanh chóng tiếp cận và dễ dàng trải nghiệm ngay từ lần chơi đầu tiên.

Trò chơi được thiết kế với nhiều chế độ chơi khác nhau như Story Mode, Endless Mode, Boss Mode và Collection Mode nhằm tăng tính đa dạng và kéo dài thời gian gắn bó của người chơi. Đồng thời, hệ thống nâng cấp và phần

thường phong phú (vũ khí, xe, đạn, nhiệm vụ hằng ngày, Lucky Spin, Daily Gift...) tạo động lực để người chơi quay lại thường xuyên, từ đó gia tăng tính hấp dẫn và khả năng duy trì lâu dài.

Về mặt kỹ thuật, đề tài khai thác tối đa sức mạnh của Unity Engine kết hợp với ngôn ngữ lập trình C# để xây dựng gameplay, trí tuệ nhân tạo (AI) cho zombie, cơ chế nâng cấp và hệ thống lưu trữ dữ liệu người chơi. Công cụ Spine Animation được sử dụng để tạo hoạt ảnh mượt mà, sinh động, đồng thời tối ưu dung lượng tài nguyên. Bên cạnh đó, trò chơi được tích hợp hệ thống quảng cáo và mua hàng trong ứng dụng (IAP), phù hợp với mô hình kinh doanh phổ biến của dòng game Hyper-Casual.

Về ý nghĩa thực tiễn, sản phẩm “Shooting Zombie” không chỉ mang tính giải trí mà còn đóng vai trò là một minh chứng cho việc vận dụng hiệu quả kiến thức đã học vào thực tế. Qua quá trình nghiên cứu và phát triển sản phẩm, nhóm thực hiện tích lũy thêm kinh nghiệm trong thiết kế gameplay, quản lý dữ liệu, tối ưu hiệu năng và triển khai game trên nền tảng Android. Đây cũng sẽ là nền tảng quan trọng để nghiên cứu và phát triển các sản phẩm game phức tạp hơn trong tương lai, chẳng hạn như game 3D, game nhiều người chơi (multiplayer) hoặc các ứng dụng gắn liền với công nghệ VR/AR.

1.5. Nội dung chính của báo cáo

Báo cáo này tập trung trình bày chi tiết toàn bộ quá trình nghiên cứu, thiết kế, xây dựng và triển khai sản phẩm game, phản ánh đầy đủ các bước từ giai đoạn hình thành ý tưởng, nghiên cứu cơ sở lý thuyết, phân tích yêu cầu, thiết kế hệ thống, phát triển sản phẩm, tối ưu hóa hiệu suất đến khâu hoàn thiện và định hướng phát triển trong tương lai.

Trước hết, báo cáo mở đầu với phần giới thiệu tổng quan về đề tài, trong đó làm rõ mục tiêu nghiên cứu, phạm vi triển khai, cũng như lý do lựa chọn đề tài. Phần này giúp người đọc nắm bắt bối cảnh nghiên cứu, giá trị thực tiễn và ý nghĩa khoa học của sản phẩm được xây dựng.

Tiếp theo, cơ sở lý thuyết được trình bày nhằm cung cấp nền tảng kiến thức chuyên môn vững chắc cho quá trình phát triển sản phẩm. Nội dung bao gồm khái niệm và vai trò của Game Engine, đặc biệt nhấn mạnh vào Unity –

công cụ chính được sử dụng trong đề tài; tổng quan về lập trình game 2D/3D, kỹ thuật mô hình hóa vật thể, xử lý va chạm; cùng các nguyên lý và phương pháp phát triển phần mềm hiện đại như lập trình hướng đối tượng (OOP), các mẫu thiết kế phần mềm (Design Pattern) và nguyên lý SOLID, đóng vai trò quan trọng trong việc xây dựng một hệ thống game có tính ổn định, dễ bảo trì và mở rộng.

Phân phân tích và thiết kế hệ thống tập trung làm rõ yêu cầu chức năng và phi chức năng của trò chơi, từ đó đưa ra mô hình luồng hoạt động, hệ thống nhân vật, gameplay, cơ chế vũ khí, trí tuệ nhân tạo (AI) và hệ thống nâng cấp. Bên cạnh đó, cấu trúc dữ liệu cũng được nghiên cứu và xây dựng bài bản thông qua việc sử dụng ScriptableObject, kết hợp với các tệp cấu hình nhằm tối ưu hóa quy trình quản lý dữ liệu trong game.

Báo cáo tiếp tục trình bày quy trình phát triển game một cách hệ thống, bao gồm việc lựa chọn công nghệ và công cụ hỗ trợ như Unity, Spine, DOTween, Odin, Git; quản lý phiên bản phần mềm và tổ chức công việc theo mô hình phân nhánh (branch); đồng thời triển khai tích hợp quảng cáo và hệ thống thanh toán trong ứng dụng (In-App Purchase – IAP) với nền tảng IronSource. Quy trình đóng gói và phát hành sản phẩm cũng được đề cập, bao gồm các bước xây dựng và thử nghiệm trên nhiều môi trường khác nhau như APK (Android), TestFlight (iOS) và Google Play.

Ở giai đoạn triển khai và xây dựng sản phẩm, báo cáo mô tả chi tiết quá trình hiện thực hóa thiết kế thành sản phẩm game hoàn chỉnh. Các chế độ chơi như Endless Mode, Boss Mode và Collection Mode được phát triển nhằm đa dạng hóa trải nghiệm người chơi. Quá trình tối ưu hiệu suất hệ thống được chú trọng với việc áp dụng kỹ thuật Object Pooling và quản lý tài nguyên hiệu quả, từ đó giảm tải bộ nhớ và nâng cao hiệu suất xử lý. Ngoài ra, trò chơi trải qua nhiều vòng thử nghiệm và điều chỉnh để đảm bảo chất lượng gameplay, sự cân bằng và tính hấp dẫn.

Kết quả đạt được thể hiện ở việc hoàn thiện sản phẩm đúng với mục tiêu ban đầu, áp dụng thành công các nguyên lý lập trình chuẩn mực, giúp sản phẩm

có khả năng bảo trì, mở rộng trong tương lai, đồng thời đáp ứng các yêu cầu kỹ thuật để phát hành chính thức trên nền tảng di động.

Cuối cùng, báo cáo đưa ra phần đánh giá tổng quan về sản phẩm đã hoàn thiện, chỉ ra các ưu điểm, hạn chế, đồng thời đề xuất định hướng phát triển tiếp theo. Các định hướng này bao gồm việc mở rộng số lượng chế độ chơi, phát triển đa nền tảng (PC, Mobile, WebGL), nâng cao chất lượng trí tuệ nhân tạo (AI) và xây dựng các cơ chế gameplay nâng cao, góp phần hoàn thiện sản phẩm và nâng cao giá trị thương mại trong tương lai.

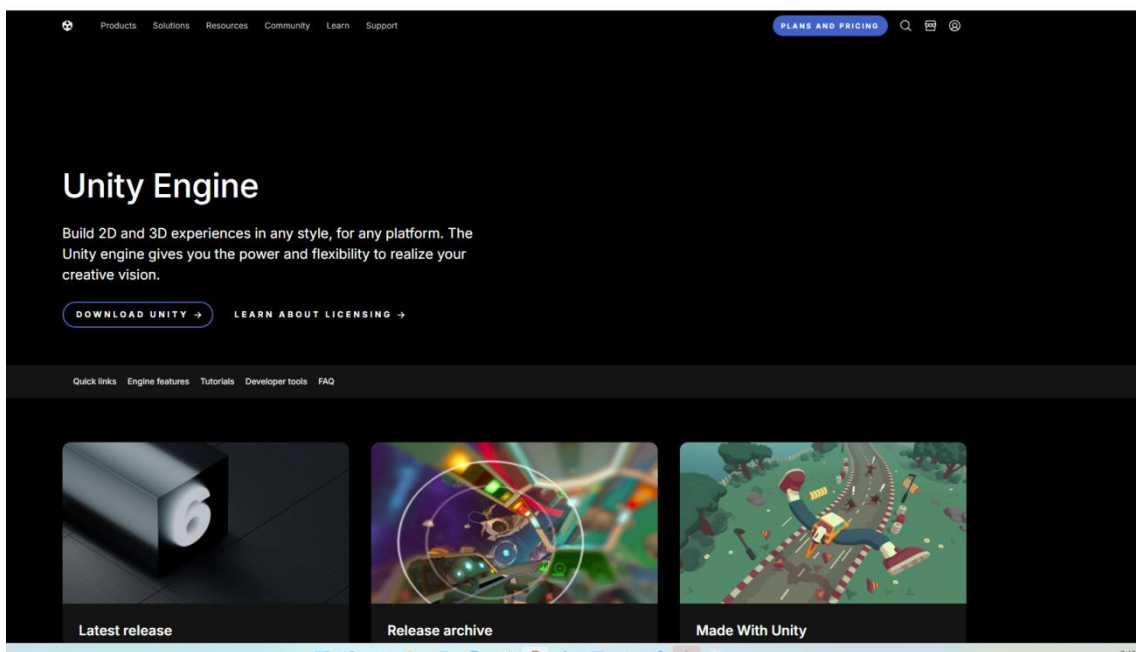
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Giới thiệu Unity Engine

2.1.1. Tổng quan về Unity Engine

Unity là một trong những game engine (công cụ phát triển game) mạnh mẽ và phổ biến nhất trên thế giới hiện nay. Được phát triển bởi Unity Technologies, Unity không chỉ là một phần mềm mà là một hệ sinh thái toàn diện, cung cấp cho các nhà phát triển bộ công cụ mạnh mẽ để xây dựng các trò chơi điện tử và các trải nghiệm tương tác khác.

Với triết lý "dân chủ hóa việc phát triển game", Unity đã phá vỡ rào cản kỹ thuật và chi phí, giúp hàng triệu nhà phát triển từ cá nhân, studio độc lập (indie) cho đến các công ty lớn có thể biến ý tưởng sáng tạo của mình thành hiện thực.



Hình 2.1. Giao diện trang chủ Unity

2.1.2. Lịch sử và tầm ảnh hưởng

Unity lần đầu tiên ra mắt vào năm 2005 tại hội nghị các nhà phát triển toàn cầu của Apple (WWDC), khi đó chỉ được biết đến như một công cụ phát triển trò chơi dành riêng cho hệ điều hành Mac OS. Tuy nhiên, nhờ định hướng chiến lược đa nền tảng, Unity nhanh chóng mở rộng và hiện nay đã hỗ trợ hơn 25 nền tảng khác nhau, bao gồm máy tính để bàn (PC, Mac, Linux), thiết bị di động (iOS, Android), các hệ máy console (PlayStation, Xbox, Nintendo Switch), nền

tăng web (WebGL), cũng như nhiều thiết bị thực tế ảo (VR) và thực tế tăng cường (AR) như Oculus, HTC Vive, và HoloLens. Theo số liệu thống kê, hơn một nửa số trò chơi trên toàn cầu, đặc biệt trong lĩnh vực game di động, được xây dựng bằng Unity. Nhiều tựa game nổi tiếng như *Genshin Impact*, *Pokémon GO*, *Call of Duty: Mobile* hay *Cuphead* chính là minh chứng rõ rệt cho sức mạnh, tính linh hoạt và tầm ảnh hưởng của engine này trong ngành công nghiệp trò chơi điện tử.

2.1.3. Các tính năng cốt lõi và nổi bật

Sức mạnh của Unity đến từ sự kết hợp hài hòa giữa nhiều tính năng ưu việt, giúp công cụ này trở thành một trong những game engine phổ biến và linh hoạt nhất hiện nay. Trước hết, Unity Editor đóng vai trò là trung tâm điều khiển chính của toàn bộ quá trình phát triển trò chơi. Đây là một trình biên tập trực quan với giao diện đồ họa mạnh mẽ, cho phép nhà phát triển dễ dàng kéo-thả (drag-and-drop) các tài sản như mô hình 3D, hình ảnh 2D hay âm thanh vào màn chơi (scene). Unity Editor hỗ trợ thiết kế màn chơi theo thời gian thực, cho phép sắp xếp, tinh chỉnh vị trí, xoay và thay đổi kích thước đối tượng ngay trong không gian 2D hoặc 3D. Đặc biệt, người dùng có thể kiểm tra và gỡ lỗi nhanh chóng bằng cách nhấn nút “Play” để trải nghiệm trò chơi ngay trong trình biên tập mà không cần phải biên dịch lại toàn bộ dự án.

Một điểm nổi bật khác là kiến trúc dựa trên Component (Component-Based Architecture), được xem là triết lý thiết kế đột phá của Unity. Trong mô hình này, mọi đối tượng trong game (GameObject) về cơ bản chỉ là một “vật chứa” rỗng, và các chức năng hay hành vi của nó được định nghĩa thông qua việc gắn các component khác nhau. Ví dụ, để một khối lập phương rơi theo trọng lực, người lập trình chỉ cần gắn component Rigidbody; để khối này có thể va chạm, cần thêm Box Collider; và để định nghĩa hành vi riêng, lập trình viên chỉ việc đính kèm một script viết bằng C#. Cách tiếp cận này mang lại sự linh hoạt cao, dễ tái sử dụng, và giúp các nhà phát triển kết hợp nhiều chức năng sáng tạo mà không bị ràng buộc cứng nhắc.

Bên cạnh đó, Unity nổi bật với khả năng hỗ trợ đa nền tảng vượt trội, hiện thực hóa phương châm “viết một lần, triển khai mọi nơi” (Build once, deploy

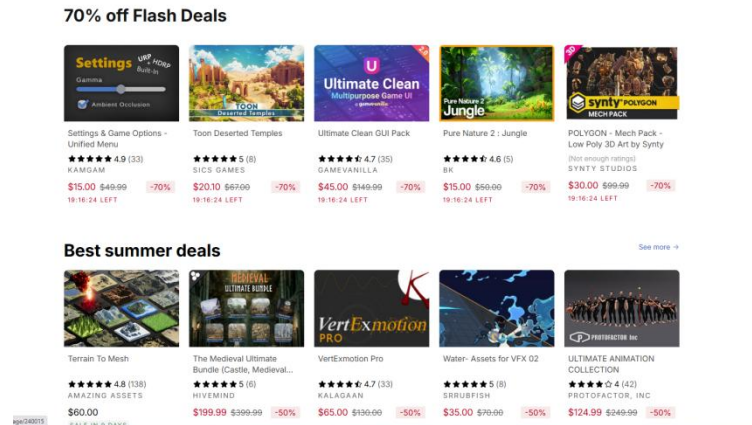
anywhere). Với một dự án duy nhất, các nhà phát triển có thể xuất bản sản phẩm của mình lên nhiều nền tảng khác nhau như PC, console, di động hay web mà không cần chỉnh sửa quá nhiều, từ đó tiết kiệm đáng kể thời gian và chi phí.

Về mặt lập trình, Unity sử dụng C# làm ngôn ngữ chính. C# là một ngôn ngữ hiện đại, mạnh mẽ, dễ tiếp cận và hoàn toàn hướng đối tượng, đồng thời tận dụng được sức mạnh của nền tảng .NET để cung cấp bộ công cụ lập trình an toàn và hiệu quả. Sự lựa chọn này khiến Unity trở nên phù hợp với cả những lập trình viên mới bắt đầu lẫn các chuyên gia phát triển game chuyên sâu.

Một trong những yếu tố làm nên sức hút lớn của Unity là Unity Asset Store, kho tài nguyên số khổng lồ được ví như một “thư viện vàng” của các nhà phát triển. Tại đây, người dùng có thể tìm thấy đủ loại tài nguyên, từ mô hình 3D, nhân vật, hoạt ảnh, hiệu ứng âm thanh, nhạc nền, cho tới các hệ thống AI, UI, gameplay hoàn chỉnh, cũng như các công cụ và tiện ích mở rộng. Điều này đặc biệt hữu ích với các đội phát triển nhỏ hoặc độc lập (indie), giúp họ rút ngắn đáng kể thời gian và công sức phát triển sản phẩm mà vẫn đảm bảo chất lượng cao.

Không chỉ mạnh mẽ ở mảng đồ họa 3D vốn là điểm khởi đầu, Unity còn cung cấp hệ thống hỗ trợ đồ họa 2D toàn diện. Engine này sở hữu các bộ công cụ vật lý tách biệt cho 2D và 3D, hệ thống tạo animation linh hoạt, cùng các giải pháp render tiên tiến như Universal Render Pipeline (URP) và High Definition Render Pipeline (HDRP). Những công cụ này cho phép nhà phát triển tạo ra đa dạng phong cách đồ họa, từ tối giản, cách điệu đến siêu thực, đáp ứng nhu cầu của nhiều thể loại game khác nhau.

Nhờ sự kết hợp của trình biên tập trực quan, kiến trúc component linh hoạt, khả năng triển khai đa nền tảng, ngôn ngữ lập trình C# hiện đại, kho tài nguyên phong phú và công nghệ đồ họa tiên tiến, Unity đã khẳng định vị thế là một trong những game engine mạnh mẽ và phổ biến nhất trên thế giới, được cả cộng đồng lập trình viên mới lẫn các studio chuyên nghiệp tin dùng.



Hình 2.2. Unity Asset Store

2.1.4. Ứng dụng Unity trong mô phỏng, kiến trúc và công nghiệp

Mặc dù Unity được biết đến chủ yếu như một công cụ phát triển trò chơi điện tử, song sự linh hoạt và khả năng tùy biến mạnh mẽ của nó đã giúp nền tảng này mở rộng tầm ảnh hưởng sang nhiều lĩnh vực công nghiệp khác. Trong ngành điện ảnh và hoạt hình, Unity được sử dụng để tạo ra các đoạn phim cắt cảnh (cutscene), hiệu ứng hình ảnh (VFX) phức tạp, và thậm chí là các bộ phim hoạt hình hoàn chỉnh được dựng trong thời gian thực, giúp rút ngắn đáng kể quy trình sản xuất. Trong lĩnh vực kiến trúc, kỹ thuật và xây dựng (AEC), Unity đóng vai trò quan trọng trong việc xây dựng các mô hình 3D tương tác, hỗ trợ khách hàng và kỹ sư dễ dàng hình dung và đánh giá thiết kế công trình trước khi thi công.

Không chỉ vậy, trong ngành ô tô và giao thông vận tải, Unity được áp dụng để thiết kế và mô phỏng các phương tiện, đồng thời tạo ra những trải nghiệm lái xe thực tế ảo phục vụ mục đích đào tạo hoặc quảng bá sản phẩm. Ngoài ra, trong giáo dục và đào tạo, engine này được khai thác để phát triển các ứng dụng mô phỏng phức tạp, chẳng hạn như mô phỏng phẫu thuật trong y tế, huấn luyện quân sự, hay các chương trình đào tạo kỹ thuật chuyên sâu.

Nhờ tính linh hoạt vượt trội, Unity không chỉ giới hạn vai trò trong ngành công nghiệp game mà còn trở thành công cụ đa năng, góp phần đổi mới quy trình sản xuất, thiết kế và đào tạo trong nhiều lĩnh vực khác nhau.

2.1.5. Cộng đồng Unity tại Việt Nam và trên thế giới

Unity sở hữu một trong những cộng đồng người dùng lớn và năng động nhất thế giới. Các diễn đàn chính thức, kênh Discord, và các nhóm trên mạng xã hội luôn sôi nổi, sẵn sàng giúp đỡ người mới.

Tại Việt Nam, cộng đồng Unity cũng rất phát triển. Nhiều studio game lớn và nhỏ đều sử dụng Unity làm công cụ chính. Unity Technologies cũng thường xuyên tổ chức các sự kiện như "Grow with Unity" và tham gia các diễn đàn game lớn tại Việt Nam, cho thấy sự cam kết và hỗ trợ mạnh mẽ đối với thị trường đầy tiềm năng này. Điều này tạo ra một môi trường thuận lợi để các nhà phát triển Việt Nam học hỏi, giao lưu và phát triển sự nghiệp

2.1.6. Kết luận

Unity là một công cụ đa năng, mạnh mẽ và dễ tiếp cận, đã thay đổi bộ mặt của ngành công nghiệp game và các lĩnh vực sáng tạo khác. Với bộ tính năng toàn diện, khả năng hỗ trợ đa nền tảng xuất sắc và một cộng đồng khổng lồ, Unity chắc chắn sẽ tiếp tục là một nền tảng hàng đầu, giúp các nhà phát triển biến những ý tưởng táo bạo nhất thành những sản phẩm ấn tượng trong tương lai.

2.2. Giới thiệu ngôn ngữ lập trình C#

2.2.1. Giới thiệu chung về C#

C# là một ngôn ngữ lập trình hiện đại được Microsoft giới thiệu vào năm 2000 như một phần trong sáng kiến .NET. Ngôn ngữ này được thiết kế bởi Anders Hejlsberg, một kiến trúc sư phần mềm nổi tiếng, người cũng là tác giả của Turbo Pascal và Delphi. Mục tiêu của C# là kết hợp sức mạnh, khả năng kiểm soát và hiệu suất cao của những ngôn ngữ như C++ với sự đơn giản, dễ tiếp cận và tốc độ phát triển nhanh của các ngôn ngữ như Visual Basic. Nhờ vậy, C# nhanh chóng trở thành một trong những ngôn ngữ lập trình chủ lực trong hệ sinh thái .NET, đồng thời được ứng dụng rộng rãi trong nhiều lĩnh vực lập trình hiện đại.

2.2.2. Đặc điểm nổi bật của C#

C# được đánh giá cao nhờ nhiều đặc điểm nổi trội. Trước hết, đây là một ngôn ngữ lập trình hiện đại, được xây dựng dựa trên nguyên tắc lập trình hướng đối tượng (OOP) với đầy đủ các khái niệm quan trọng như đóng gói, kế thừa,

trừu tượng và đa hình. Điều này giúp tổ chức mã nguồn một cách logic, dễ bảo trì, mở rộng và tái sử dụng.

Bên cạnh đó, C# là một ngôn ngữ an toàn về kiểu dữ liệu (type-safe), yêu cầu lập trình viên khai báo rõ ràng kiểu dữ liệu cho biến, từ đó giúp trình biên dịch phát hiện và ngăn chặn các lỗi tiềm ẩn ngay từ quá trình viết mã, giảm thiểu lỗi khi chương trình chạy. Một ưu điểm quan trọng khác của C# là cơ chế quản lý bộ nhớ tự động thông qua Garbage Collector (GC), giúp lập trình viên không cần tự quản lý việc cấp phát và giải phóng bộ nhớ như trong C++, đồng thời hạn chế tình trạng rò rỉ bộ nhớ.

Không chỉ mạnh mẽ và an toàn, C# còn rất linh hoạt và đa năng. Nhờ sự phát triển của .NET Core (nay là .NET), C# trở thành ngôn ngữ đa nền tảng, được ứng dụng để xây dựng nhiều loại phần mềm, bao gồm ứng dụng desktop (Windows Forms, WPF), ứng dụng web (ASP.NET), ứng dụng di động (Xamarin, .NET MAUI), dịch vụ đám mây, và đặc biệt là trò chơi điện tử.

2.2.3. Sự liên quan mật thiết giữa C# và Unity

Trong quá khứ, Unity từng hỗ trợ nhiều ngôn ngữ kịch bản khác nhau như Boo hay UnityScript, tuy nhiên hiện nay C# đã trở thành ngôn ngữ lập trình duy nhất được hỗ trợ chính thức. Sự lựa chọn này xuất phát từ cả chiến lược phát triển của Unity lẫn lợi ích rõ rệt cho cộng đồng lập trình viên.

C# mang lại sự cân bằng tối ưu giữa tính dễ học và sức mạnh lập trình. Với cú pháp rõ ràng, trực quan và dễ đọc hơn so với C++, C# phù hợp với cả người mới bắt đầu và các lập trình viên chuyên nghiệp phát triển những hệ thống trò chơi phức tạp. Unity sử dụng Mono và gần đây là IL2CPP để biên dịch mã C# thành mã máy gốc, đảm bảo hiệu suất vận hành cao – một yêu cầu thiết yếu của các trò chơi điện tử.

Ngôn ngữ này cũng được tích hợp hoàn hảo với Unity Editor. Unity cung cấp một hệ thống API mạnh mẽ, cho phép các đoạn mã C# (scripts) điều khiển trực tiếp các thành phần trong trò chơi. Nhà phát triển chỉ cần kéo và thả một script vào đối tượng (GameObject) để lập tức gán cho nó một hành vi mới. Sự kết hợp giữa Unity và C# còn được củng cố nhờ cộng đồng phát triển rộng lớn. Với nền tảng C# phổ biến từ Microsoft cùng cộng đồng người dùng Unity toàn

cầu, lập trình viên dễ dàng tiếp cận tài liệu, thư viện, plugin và nhận được hỗ trợ nhanh chóng khi gặp khó khăn.

Nhờ những đặc điểm này, C# không chỉ là công cụ lập trình chính thức của Unity mà còn đóng vai trò là cầu nối giúp Unity trở thành một nền tảng phát triển trò chơi mạnh mẽ, linh hoạt và dễ tiếp cận với mọi đối tượng lập trình viên.

2.3. Giới thiệu thể loại game hyper-casual

2.3.1. Giới thiệu chung

Hyper-casual (tạm dịch: siêu phổ thông hoặc siêu đơn giản) là một thể loại game di động đã bùng nổ và thống trị các bảng xếp hạng lượt tải về trong nhiều năm gần đây. Đúng như tên gọi, đặc điểm cốt lõi của thể loại này là sự tối giản đến mức tối đa, từ lối chơi, đồ họa cho đến giao diện người dùng.

Mục tiêu của game Hyper-casual là tạo ra một trải nghiệm "chơi ngay lập tức", thu hút người chơi chỉ trong vài giây đầu tiên và giữ chân họ bằng các vòng lặp gameplay gây nghiện.

2.3.2. Các đặc điểm cốt lõi

Để một trò chơi điện tử được xếp vào thể loại hyper-casual, sản phẩm đó thường phải đáp ứng một số đặc điểm cơ bản đặc trưng cho dòng game này. Trước hết, cơ chế chơi cần được thiết kế tối giản, thường chỉ tập trung vào một hoặc hai hành động chính được lặp đi lặp lại, chẳng hạn như chạm để nhảy, vượt để di chuyển hoặc giữ để tăng tốc. Đặc điểm này giúp người chơi dễ dàng tiếp cận ngay từ lần đầu trải nghiệm, không đòi hỏi hướng dẫn phức tạp hay hệ thống tutorial chi tiết.

Bên cạnh cơ chế điều khiển đơn giản, phong cách đồ họa và thiết kế giao diện của game hyper-casual cũng hướng đến sự tối giản. Giao diện người dùng (UI) thường được tinh gọn, chỉ bao gồm màn hình chơi chính và một số nút chức năng cơ bản như “chơi lại” (retry), hạn chế tối đa các menu hoặc hệ thống phức tạp. Phong cách đồ họa của thể loại này chủ yếu sử dụng hình khối cơ bản, màu sắc tươi sáng, độ tương phản cao, giúp người chơi dễ dàng nhận diện và phân biệt các yếu tố trong trò chơi.

Một yếu tố quan trọng khác là thời lượng mỗi vòng chơi thường rất ngắn, chỉ kéo dài từ vài giây đến dưới một phút. Đặc điểm này đáp ứng nhu cầu giải trí

nhANH chóng, phù hợp với thói quen chơi game ngắn hạn của nhiều người dùng di động. Quá trình thất bại và chơi lại cũng được thiết kế mượt mà, tạo điều kiện cho người chơi dễ dàng tiếp tục thử thách nhằm đạt điểm số cao hơn.

Cuối cùng, phần lớn trò chơi hyper-casual được xây dựng theo mô hình chơi vô tận (endless gameplay), không có điểm kết thúc cụ thể. Mục tiêu chính của người chơi là liên tục vượt qua thành tích cá nhân hoặc so sánh điểm số với người khác. Cơ chế vòng lặp “chơi – thua – chơi lại” được tối ưu nhằm tạo ra cảm giác hứng thú và thúc đẩy người chơi tiếp tục trải nghiệm với suy nghĩ “chỉ thêm một ván nữa”. Nhờ những yếu tố này, thể loại hyper-casual đã trở thành một trong những xu hướng nổi bật trên thị trường game di động, thu hút đông đảo người chơi nhờ tính đơn giản nhưng gây nghiện cao.



Hình 2.3. Vòng lặp game

2.3.3. Mô hình kinh doanh và kiếm tiền

Không giống như nhiều thể loại trò chơi điện tử khác thường dựa trên doanh thu từ việc bán vật phẩm trong game (In-App Purchase), mô hình kinh doanh của các trò chơi hyper-casual chủ yếu phụ thuộc vào quảng cáo trong ứng dụng (In-App Advertising). Đây là đặc điểm quan trọng giúp thể loại này tiếp cận được đông đảo người chơi, bởi trò chơi hoàn toàn miễn phí để tải xuống và trải nghiệm, từ đó thu hút lượng người dùng khổng lồ.

Trong số các hình thức quảng cáo phổ biến, quảng cáo video có thưởng (Rewarded Video Ads) là phương thức được sử dụng rộng rãi nhất. Với hình thức này, người chơi có thể lựa chọn xem một đoạn quảng cáo ngắn, thường kéo dài từ 15 đến 30 giây, để đổi lấy những lợi ích nhất định trong trò chơi, chẳng hạn như hồi sinh nhân vật sau khi thất bại, nhận thêm tiền tệ trong game hoặc mở

khóa các nhân vật, trang phục (skin) và vật phẩm đặc biệt khác. Việc xem quảng cáo mang tính tự nguyện giúp người chơi cảm thấy ít bị làm phiền hơn, đồng thời vẫn tạo ra giá trị cho nhà phát triển.

Bên cạnh đó, quảng cáo xen kẽ (Interstitial Ads) cũng là một chiến lược monetization phổ biến trong hyper-casual games. Đây là các quảng cáo toàn màn hình được hiển thị vào những khoảng nghỉ tự nhiên trong quá trình chơi, chẳng hạn sau mỗi vài lượt thất bại. Cách triển khai này giúp duy trì trải nghiệm người dùng tương đối mượt mà, đồng thời đảm bảo hiệu quả tiếp cận quảng cáo.

Mô hình kinh doanh dựa trên quảng cáo của thể loại hyper-casual vận hành dựa theo quy luật số đông: bằng cách cung cấp trò chơi miễn phí và dễ tiếp cận, nhà phát triển có thể thu hút hàng triệu lượt tải về. Dù chỉ một tỷ lệ nhỏ người dùng tương tác với quảng cáo, số lượng lớn người chơi vẫn đảm bảo nguồn doanh thu ổn định và bền vững. Chính chiến lược này đã góp phần quan trọng vào sự thành công và sức lan tỏa toàn cầu của dòng game hyper-casual trên thị trường di động.

2.3.4. Ví dụ tiêu biểu

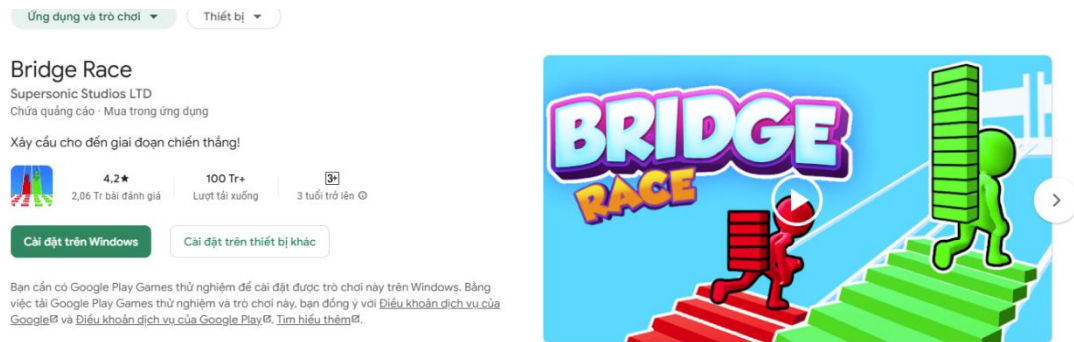
Một số tựa game hyper-casual đã đạt được thành công vang dội trên toàn cầu:



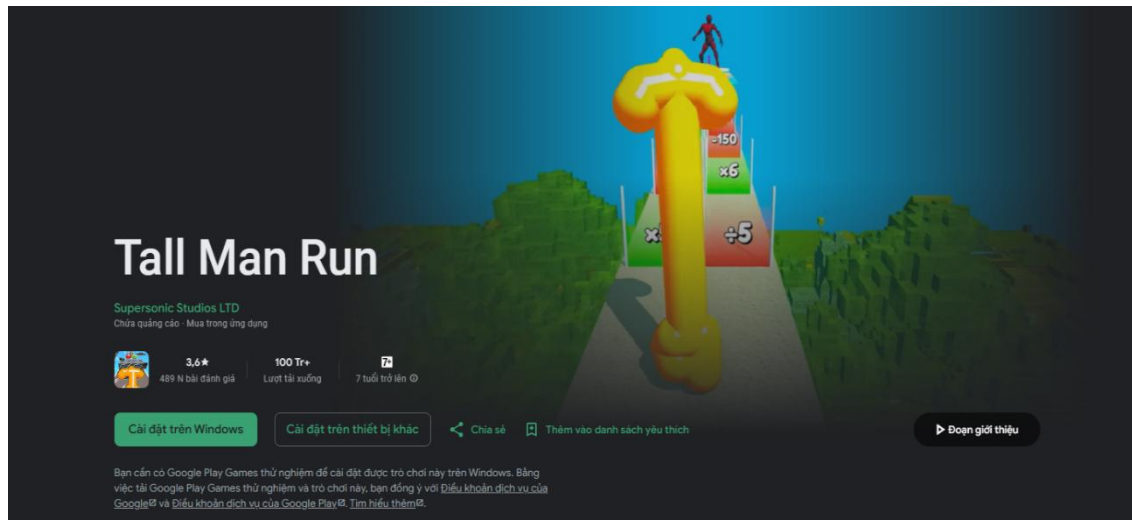
Hình 2.4. Game Helix Jump trên Google Play



Hình 2.5. Game Paper.io trên Google Play



Hình 2.6. Game Bridge Race io trên Google Play



Hình 2.7. Game Tall Man Run trên Google Play

2.3.5. Kết luận

Hyper-casual là một thể loại game đặc biệt, tập trung vào sự tinh túy của giải trí: đơn giản, nhanh chóng và lôi cuốn. Nó đã chứng minh rằng không cần đồ họa phức tạp hay cốt truyện sâu sắc, một cơ chế gameplay thông minh và gây nghiện vẫn có thể chinh phục được thị trường game di động toàn cầu.

Tuy nhiên, thể loại này cũng đối mặt với thách thức về việc giữ chân người chơi lâu dài và sự cạnh tranh khốc liệt, đòi hỏi các nhà phát triển phải liên tục sáng tạo để tìm ra ý tưởng mới mẻ và hấp dẫn.

CHƯƠNG 3: CÀI ĐẶT MÔI TRƯỜNG

3.1. Giới thiệu môi trường làm việc

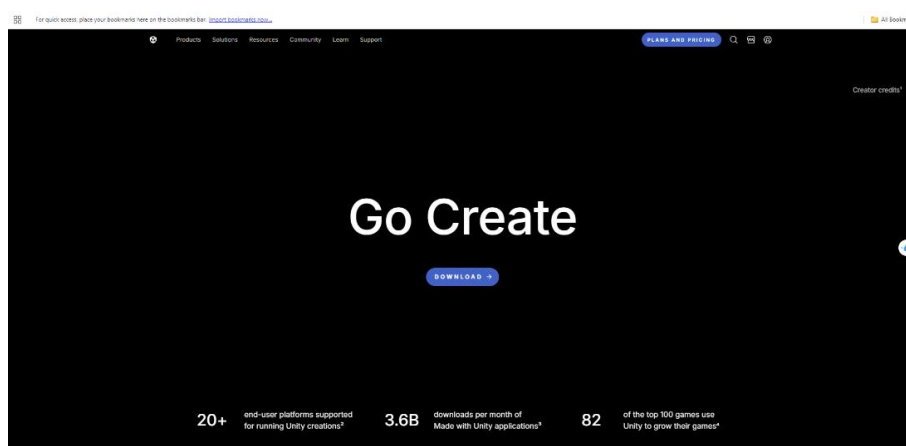
3.1.1. Mục tiêu

Mục tiêu của phần này là cài đặt và thiết lập môi trường làm việc với Unity, đồng thời làm quen với giao diện và các thành phần cơ bản trong Unity Editor. Unity là một công cụ phát triển game đa nền tảng (game engine) được sử dụng rộng rãi trong nhiều lĩnh vực như phát triển trò chơi, mô phỏng, kiến trúc và giáo dục. Công cụ này hỗ trợ lập trình cả 2D và 3D, cung cấp hệ thống UI trực quan, thư viện tài nguyên phong phú, cùng ngôn ngữ lập trình C# giúp người dùng dễ dàng xây dựng, tùy chỉnh và triển khai sản phẩm trên nhiều nền tảng khác nhau như Windows, Android, iOS và Web. Việc nắm vững cách cài đặt và thao tác cơ bản trong Unity là nền tảng quan trọng để phát triển các chức năng nâng cao trong các giai đoạn tiếp theo của dự án.

3.1.2. Hướng dẫn cài đặt Unity

Để triển khai và phát triển trò chơi, cần tiến hành cài đặt môi trường Unity và thiết lập các thành phần hỗ trợ cần thiết. Quy trình cài đặt được thực hiện như sau:

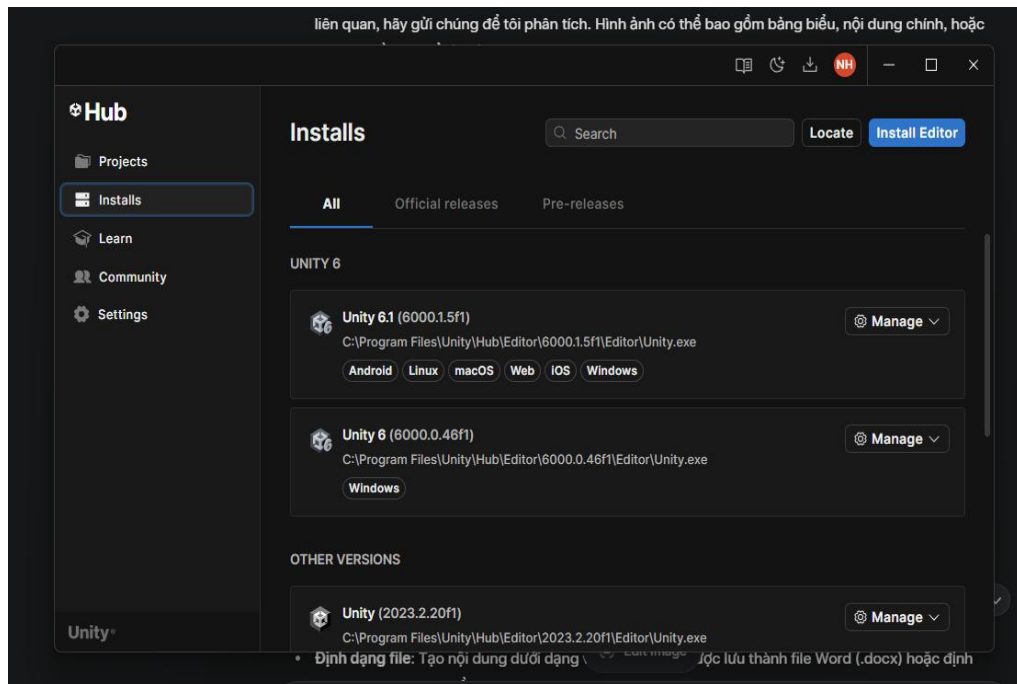
Bước 1: Truy cập trang chủ Unity tại địa chỉ: <https://unity.com/download> và tải về Unity Hub. Unity Hub là công cụ hỗ trợ quản lý phiên bản Unity Editor và các dự án đang phát triển.



Hình 3.1. Giao diện trang tải về Unity Hub.

Bước 2: Cài đặt và khởi chạy Unity Hub. Trong giao diện Unity Hub, chọn mục Installs, sau đó nhấn Add để thêm phiên bản Unity phù hợp. Trong

khóa luận này, em sử dụng phiên bản Unity 2021.3.26f1 (LTS) nhằm đảm bảo tính ổn định và khả năng hỗ trợ lâu dài.



Hình 3.2. Giao diện Installs – chọn phiên bản Unity để cài.

Bước 3: Trong quá trình cài đặt phiên bản Unity, lựa chọn bổ sung các mô-đun cần thiết như:

- Android Build Support
- SDK & NDK Tools
- OpenJDK

Các mô-đun này giúp biên dịch và triển khai trò chơi lên nền tảng Android.

Bước 4: Sau khi hoàn tất cài đặt, khởi tạo dự án mới bằng cách chọn New Project → 2D Core Template, đặt tên cho dự án và lựa chọn thư mục lưu trữ phù hợp.

Bước 5: Mở dự án và tiến hành cấu hình các thành phần cơ bản, bao gồm thiết lập giao diện làm việc trong Unity Editor, lựa chọn hệ tọa độ hiển thị, cấu hình cảnh (Scene) và hệ thống ánh sáng cơ bản theo yêu cầu của trò chơi.

3.1.3. Tạo Project mới trong Unity

Để bắt đầu phát triển trò chơi trên Unity, người dùng trước tiên cần khởi tạo một dự án mới. Quá trình thực hiện được tiến hành theo các bước sau:

Bước 1: Khởi chạy Unity Hub

Mở phần mềm Unity Hub đã được cài đặt trên máy tính. Tại giao diện chính, chọn mục Projects trong thanh điều hướng bên cạnh để xem danh sách các dự án hiện có.

Bước 2: Tạo dự án mới

Nhấn nút New Project (Tạo dự án mới). Lúc này, Unity Hub sẽ hiển thị danh sách các mẫu (template) dùng để khởi tạo dự án.

Bước 3: Lựa chọn Template phù hợp

Tại cửa sổ tạo dự án, lựa chọn một template phù hợp với thể loại trò chơi đang phát triển:

- 2D: dành cho các trò chơi sử dụng không gian hai chiều (ví dụ: platformer, puzzle).
- 3D: dùng cho các trò chơi với môi trường ba chiều (ví dụ: bắn súng, phiêu lưu, mô phỏng).

Việc lựa chọn template đúng ngay từ đầu giúp Unity tự động thiết lập cấu hình cảnh (scene), hệ thống ánh sáng và camera phù hợp với loại game.

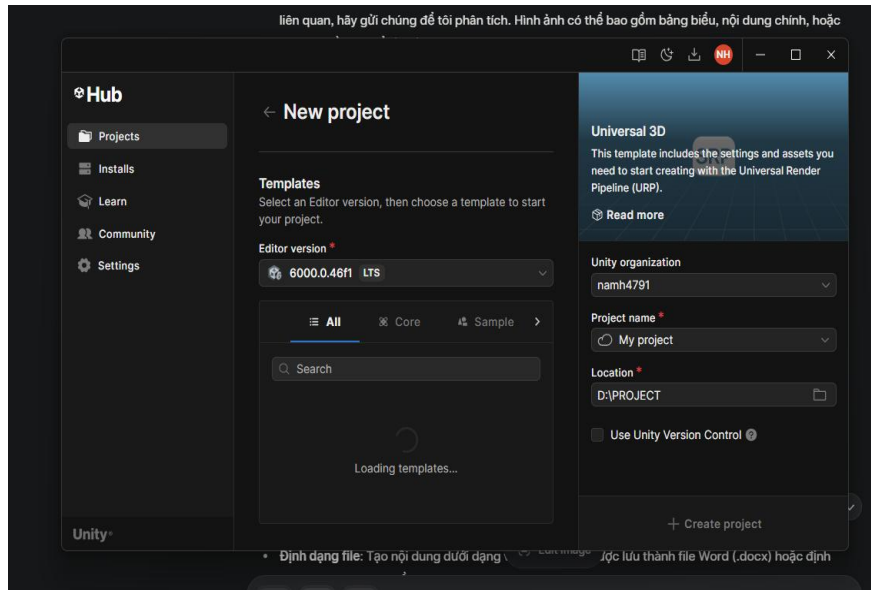
Bước 4: Đặt tên và chọn vị trí lưu dự án

Tại mục Project Name, nhập tên của dự án theo quy tắc đặt tên nhất quán (không dấu, không ký tự đặc biệt).

Trong mục Location, chọn thư mục lưu trữ dự án trên ổ đĩa để đảm bảo thuận tiện trong quản lý và sao lưu.

Bước 5: Khởi tạo dự án

Sau khi hoàn tất các thông tin trên, nhấn Create. Unity Hub sẽ tiến hành tạo cấu trúc dự án, thiết lập thư viện tài nguyên mặc định và khởi chạy Unity Editor để sẵn sàng cho quá trình phát triển.



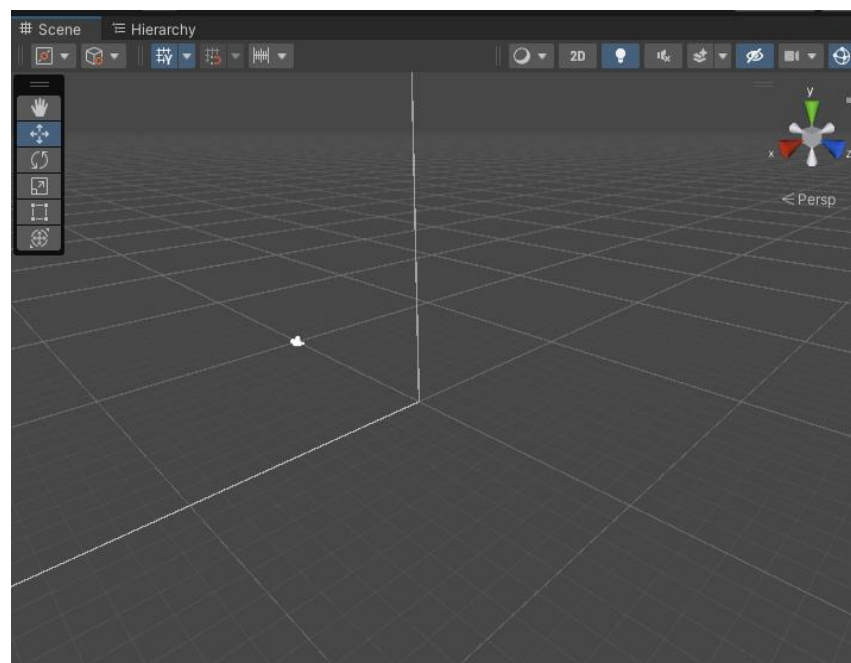
Hình 3.3. Giao diện tạo Project mới.

3.1.4. Môi trường làm việc trong Unity

Giao diện chính gồm 5 khu vực:

Khu vực 1. Cửa sổ Scene:

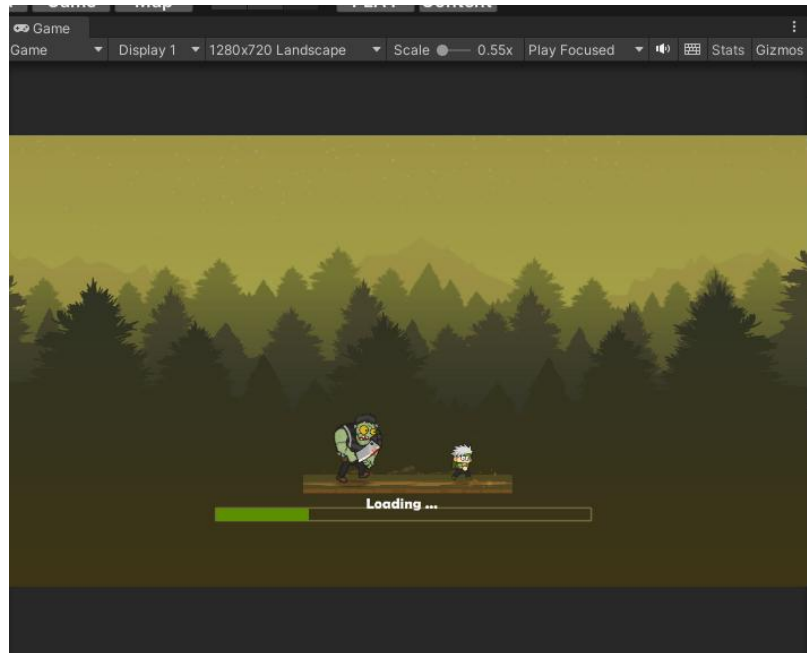
Cửa sổ Scene là nơi trực quan hóa và sắp xếp các đối tượng trong trò chơi như Camera, Light, nhân vật, vật thể môi trường,... Tại đây, người dùng có thể thực hiện các thao tác như kéo – thả, xoay, phóng to, thu nhỏ, hoặc căn chỉnh vị trí của đối tượng trong không gian làm việc.



Hình 3.4. Cửa sổ Scene

Khu vực 2. Cửa sổ Game:

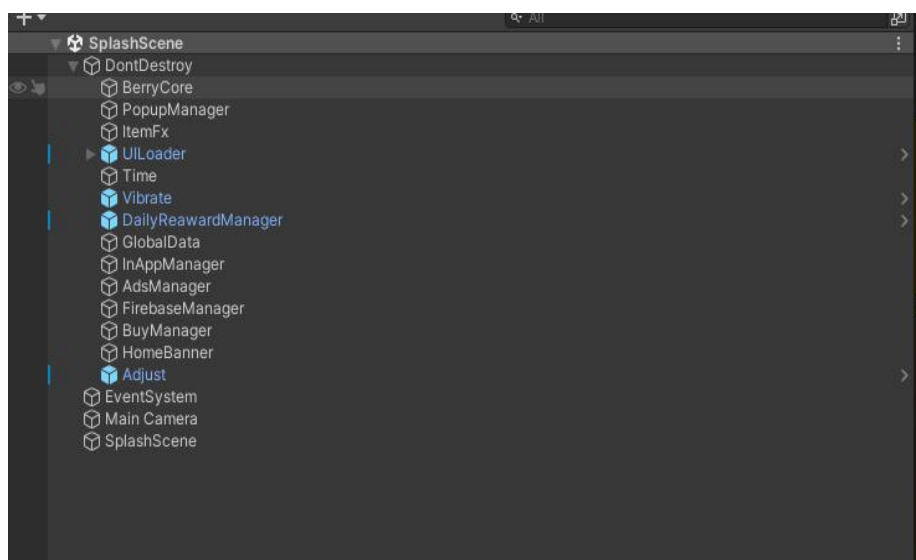
Cửa sổ Game hiển thị kết quả mô phỏng trò chơi như người chơi sẽ nhìn thấy khi chạy game. Khi nhấn nút Play, Unity sẽ mô phỏng toàn bộ hoạt động của trò chơi trong thời gian thực và thể hiện tại cửa sổ này.



Hình 3.5. Cửa sổ Game

Khu vực 3. Cửa sổ Hierarchy:

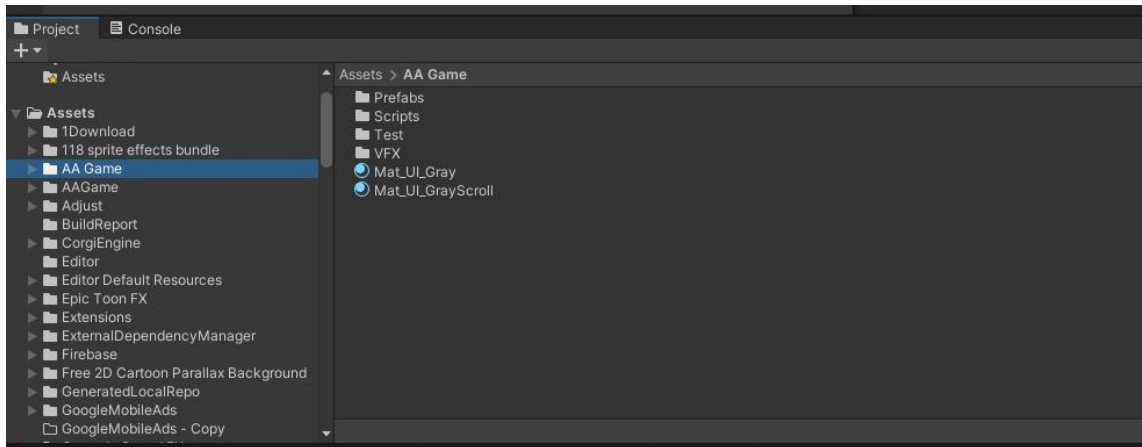
Cửa sổ Hierarchy hiển thị danh sách toàn bộ các đối tượng (GameObjects) có trong Scene hiện tại dưới dạng cấu trúc cây. Khu vực này hỗ trợ tổ chức và quản lý mối quan hệ cha – con giữa các đối tượng trong game.



Hình 3.6. Cửa sổ Hierarchy

Khu vực 4. Cửa sổ Project:

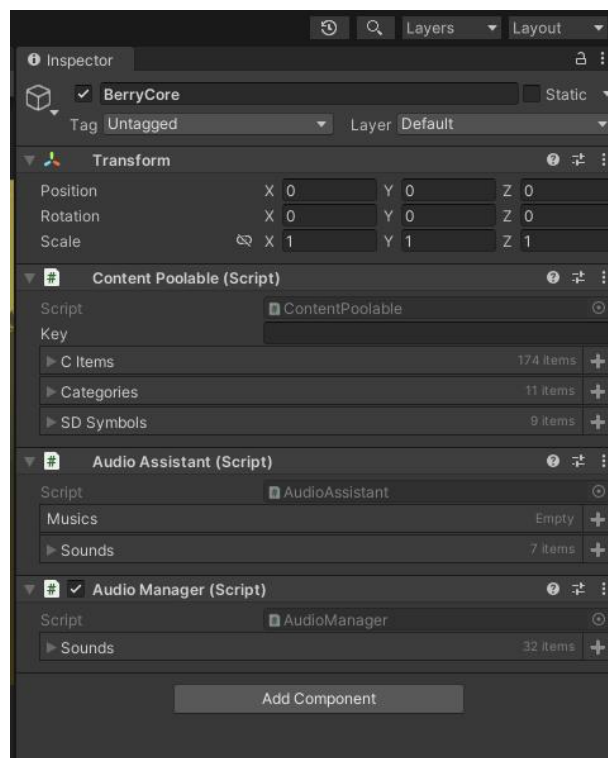
Cửa sổ Project chứa toàn bộ tài nguyên (Assets) của dự án như mô hình 3D, hình ảnh, âm thanh, đoạn mã (script), prefab, material, v.v. Đây là nơi giúp người dùng quản lý tài nguyên và tổ chức thư mục theo nhu cầu phát triển.



Hình 3.7. Cửa sổ Project

Khu vực 5. Cửa sổ Inspector:

Cửa sổ Inspector hiển thị và cho phép chỉnh sửa các thuộc tính của đối tượng đang được chọn trong Scene hoặc Hierarchy. Các thành phần như Transform, Script, Collider,... có thể được cấu hình trực tiếp trong khu vực này.



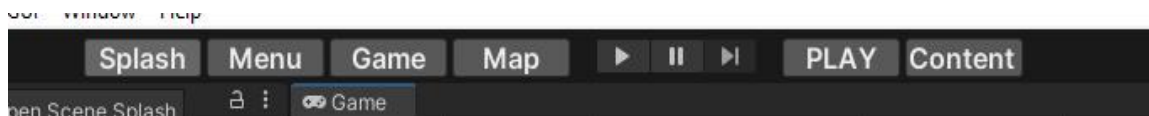
Hình 3.8. Cửa sổ Inspector

Trong quá trình phát triển, Unity cung cấp một nhóm nút điều khiển cho phép người dùng chạy thử, tạm dừng hoặc theo dõi từng bước hoạt động của trò chơi. Các nút này nằm ở phía trên giao diện Unity Editor, được minh họa tại Hình 3.10

Các chức năng bao gồm:

- Play: Dùng để chạy thử trò chơi trong môi trường Unity. Khi nhấn nút này, toàn bộ logic và chuyển động trong game sẽ được mô phỏng như khi trò chơi chạy thực tế.
- Pause: Tạm dừng quá trình chạy thử trò chơi. Trạng thái game được giữ nguyên tại thời điểm tạm dừng giúp người phát triển dễ dàng quan sát hoặc kiểm tra dữ liệu nội bộ.
- Step: Cho phép chạy từng khung hình (frame) một trong khi game đang ở chế độ tạm dừng. Tính năng này đặc biệt hữu ích khi cần kiểm tra hành vi chi tiết của các đối tượng trong game.

Ngoài ra, người dùng còn có thể chuyển đổi giữa các Scene đang mở thông qua các tab hiển thị bên cạnh nhóm nút điều khiển.



Hình 3.9. Các nút điều khiển chính

3.1.6. Một số thao tác cơ bản

Trong quá trình làm việc với giao diện Scene, người dùng thường xuyên phải thao tác với camera và các đối tượng (GameObjects). Dưới đây là một số thao tác cơ bản cần nắm:

- Di chuyển góc nhìn (Camera View):
Giữ chuột phải và sử dụng các phím W, A, S, D để di chuyển camera trong không gian làm việc.
- Di chuyển đối tượng (Move):
Chọn đối tượng và nhấn phím W để kích hoạt công cụ Move. Khi đó, đối tượng có thể được di chuyển theo các trục X, Y, Z.
- Xoay đối tượng (Rotate):
Nhấn phím E để sử dụng công cụ Rotate, cho phép xoay đối tượng quanh các trục tọa độ.

- Phóng to/Thu nhỏ và Focus đối tượng:
Nhấn phím F để đưa camera tập trung vào đối tượng đang chọn (*Focus*).
Sử dụng con lăn chuột (Scroll) để điều chỉnh mức thu phóng.

3.1.7. Kết luận

Sau khi hoàn thành các bước cài đặt và làm quen với Unity Editor, người học đã có thể bắt đầu xây dựng các dự án game 2D hoặc 3D cơ bản, nắm vững các khu vực chính trong giao diện Unity và thực hiện các thao tác cơ bản. Đây là nền tảng quan trọng để tiếp tục học và phát triển kỹ năng lập trình game.

CHƯƠNG 4: GIỚI THIỆU SẢN PHẨM

4.1. Giới thiệu chung

4.1.1. Tên khóa luận

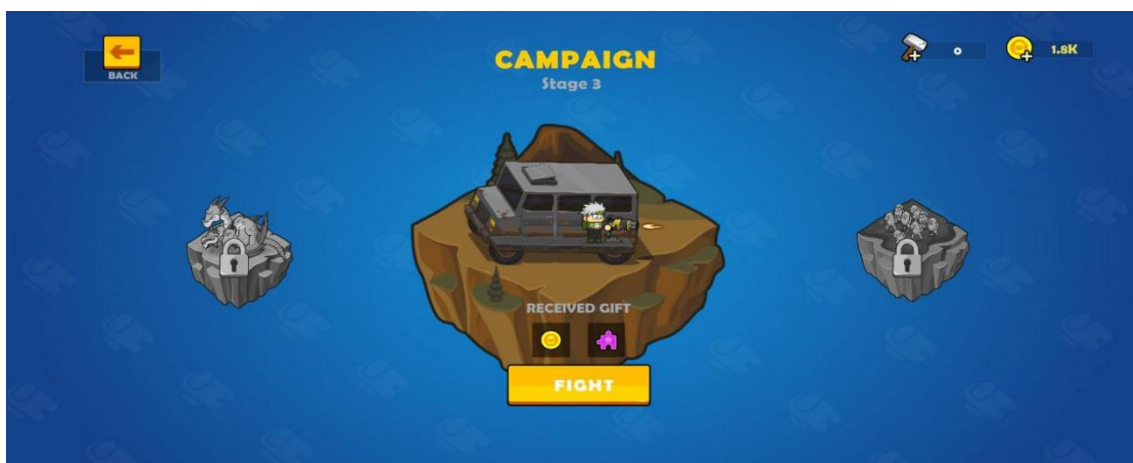
TÌM HIỂU VÀ PHÁT TRIỂN GAME HYPER-CASUAL “SHOOTING ZOMBIE” TRÊN UNITY

4.1.2. Ý tưởng và nội dung trò chơi

Trong trò chơi, người chơi điều khiển một nhân vật ngồi trên xe ô tô, đồng thời vừa di chuyển vừa bắn những con zombie đang truy đuổi. Zombie sẽ tiến công xe, đòi hỏi người chơi phải vận dụng kỹ năng để tiêu diệt càng nhiều zombie càng tốt. Trong quá trình chơi, người chơi có thể nâng cấp súng, xe ô tô và loại đạn, từ đó tăng sức mạnh chiến đấu và vượt qua các màn chơi ngày càng khó hơn.

4.1.3. Các chế độ chơi (Mode)

Story Mode: Theo cốt truyện, tăng dần độ khó và có boss sau mỗi 5 màn.



Hình 4.1. Chọn mode story (Campaign Mode)



Hình 4.2. Game Play Mode Story

Endless Mode: Chơi không giới hạn cho tới khi thua.

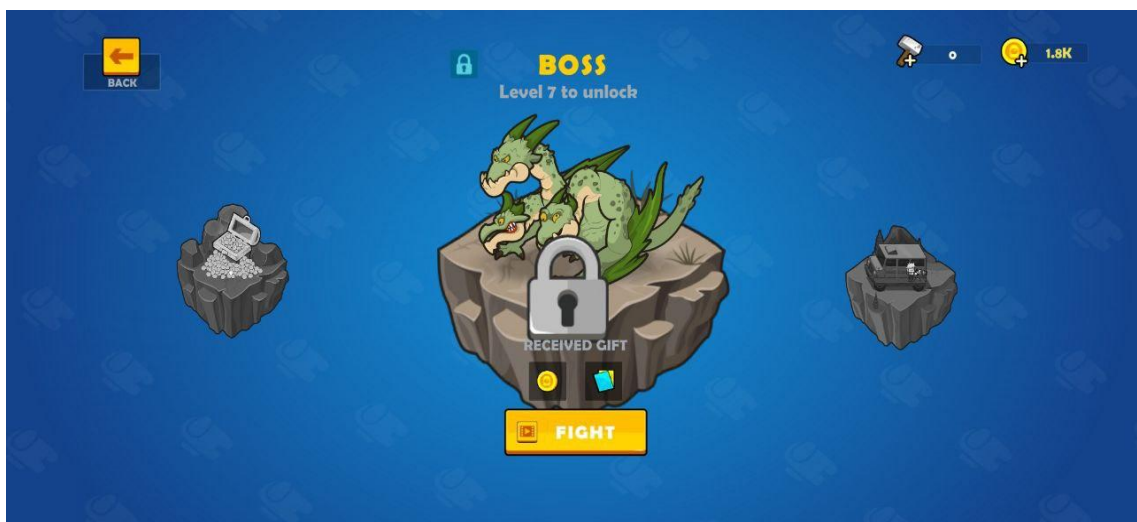


Hình 4.3. Chọn Mode Endless



Hình 4.4. Game Play Mode Endless

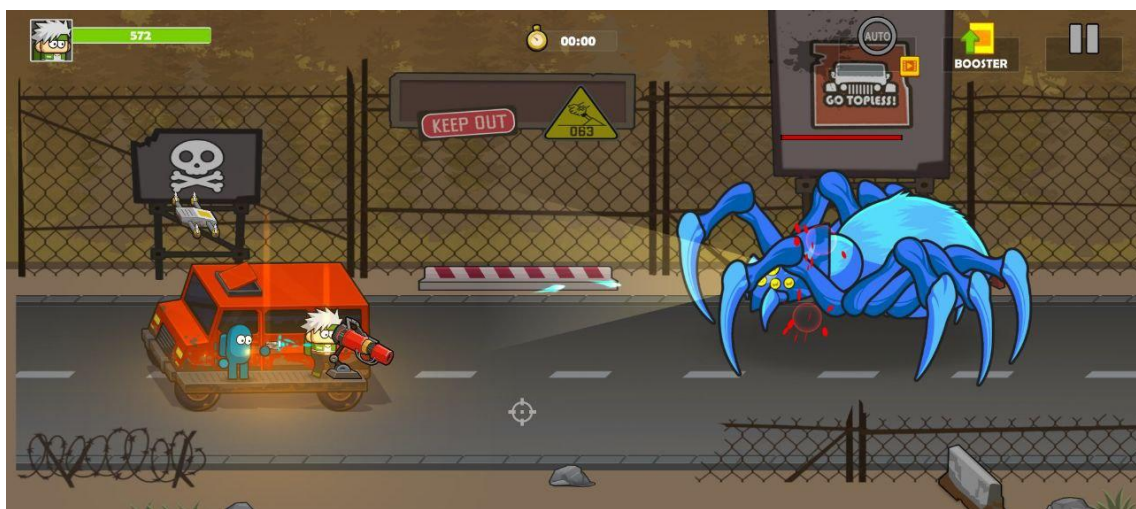
Boss Mode: Chiến đấu với boss đặc biệt để nhận phần thưởng lớn.



Hình 4.5. Chọn chơi Mode Boss

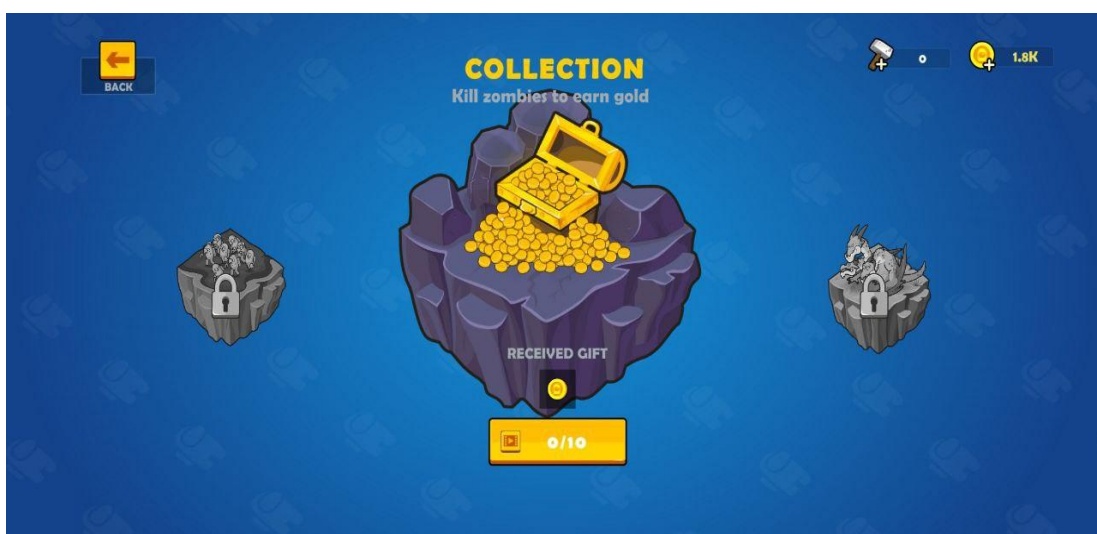


Hình 4.6. Chọn Boss chiến đấu trong mode boss



Hình 4.7. Game Play Mode Boss

Collection Mode: Người chơi truy đuổi zombie để thu thập vàng.



Hình 4.8. Chọn Mode Collection



Hình 4.9. Game play mode Collection

4.1.4. Tính năng nổi bật

Trò chơi được thiết kế với lối chơi đơn giản, người chơi chỉ cần thực hiện các thao tác chạm và kéo để điều khiển súng, giúp việc tiếp cận trò chơi trở nên dễ dàng và phù hợp với nhiều đối tượng. Giao diện người dùng được xây dựng theo hướng thân thiện và trực quan, đảm bảo trải nghiệm mượt mà cho người chơi ở nhiều độ tuổi khác nhau. Bên cạnh đó, hệ thống nâng cấp đa dạng cho phép người chơi cải thiện súng, ô tô, đạn và nhiều tính năng hỗ trợ khác, góp phần duy trì hứng thú trong suốt quá trình trải nghiệm.

Trò chơi còn tích hợp hệ thống nhiệm vụ phong phú, chiến đấu với các boss và nhiều nhiệm vụ phụ, giúp tạo nên sự đa dạng và thử thách. Cơ chế phần thưởng hấp dẫn được triển khai thông qua các hình thức như nhận quà qua cửa sổ bật lên (popup), vòng quay may mắn hay nhân đôi tài nguyên sau mỗi chiến thắng, kết hợp với tiến trình cấp độ rõ ràng nhằm khuyến khích người chơi tiếp tục khám phá. Ngoài ra, trò chơi hỗ trợ tính năng lưu trữ dữ liệu và theo dõi thông tin người chơi, đảm bảo sự liên tục và ổn định trong trải nghiệm.

4.1.5. Công nghệ và công cụ sử dụng

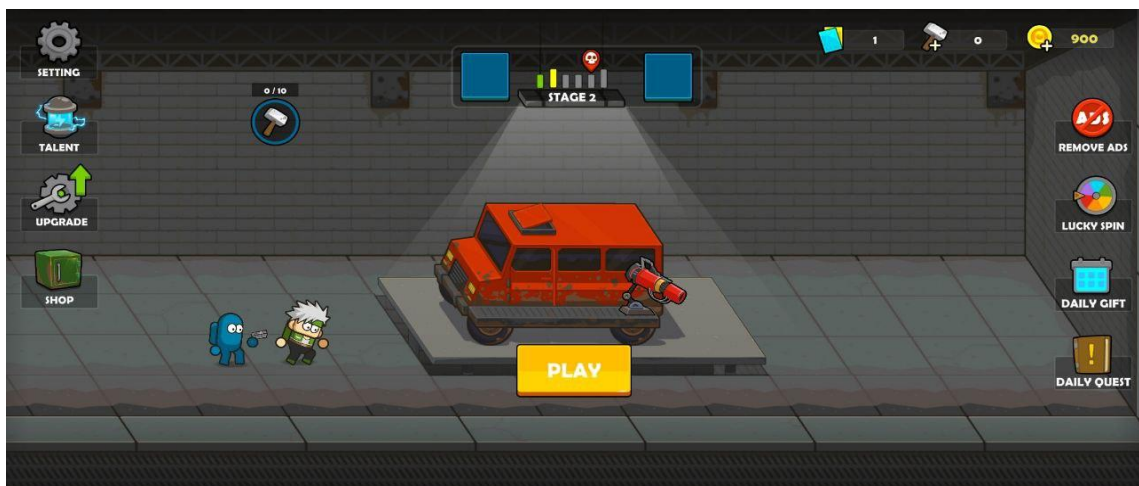
Trong quá trình phát triển trò chơi, nhóm sử dụng Unity Engine làm nền tảng chính nhờ khả năng hỗ trợ cả môi trường 2D và 3D, đồng thời cho phép xuất bản sản phẩm trên hệ điều hành Android. Ngôn ngữ lập trình được lựa chọn là C#,

giúp xây dựng các tính năng và logic của trò chơi một cách hiệu quả. Để tạo ra các chuyển động nhân vật mượt mà và sinh động, nhóm sử dụng công cụ Spine Animation.

Về kiến trúc phần mềm, dự án áp dụng nhiều mẫu thiết kế (Design Patterns) phổ biến như Singleton, Observer và State nhằm tổ chức mã nguồn rõ ràng, dễ bảo trì và mở rộng. Trong khâu quản lý mã nguồn, nhóm triển khai hệ thống Git/GitLab để lưu trữ và kiểm soát phiên bản dự án, kết hợp với công cụ Fork – một giao diện Git GUI hỗ trợ làm việc nhóm thuận tiện và hiệu quả.

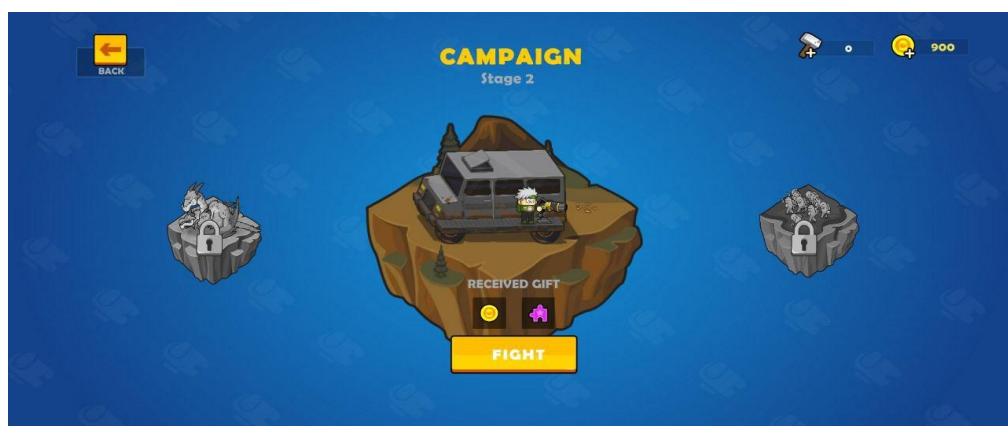
4.1.6. Thiết kế giao diện

Màn hình chính: Trạm sửa xe, có các nút như Shop, Upgrade, LuckySpin, Quest, DailyGift, Setting, Talent, ...



Hình 4.10. Màn hình chính

Màn hình chọn mode: Hiển thị phần thưởng, hình ảnh mode.



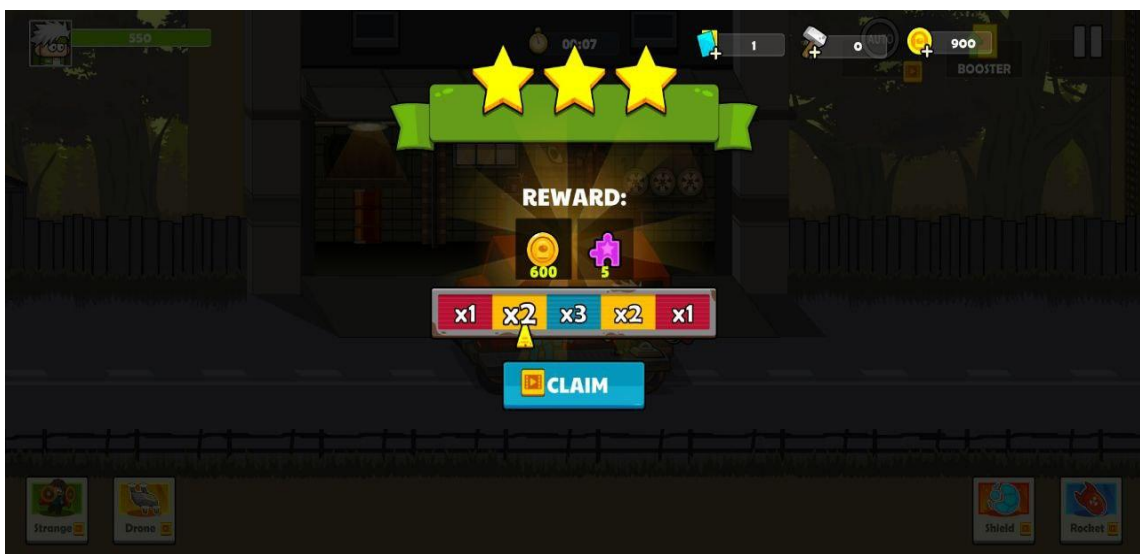
Hình 4.11. Màn hình chọn mode chơi

Màn hình gameplay: Hiệu ứng âm thanh, hiệu ứng hình ảnh sống động khi chiến

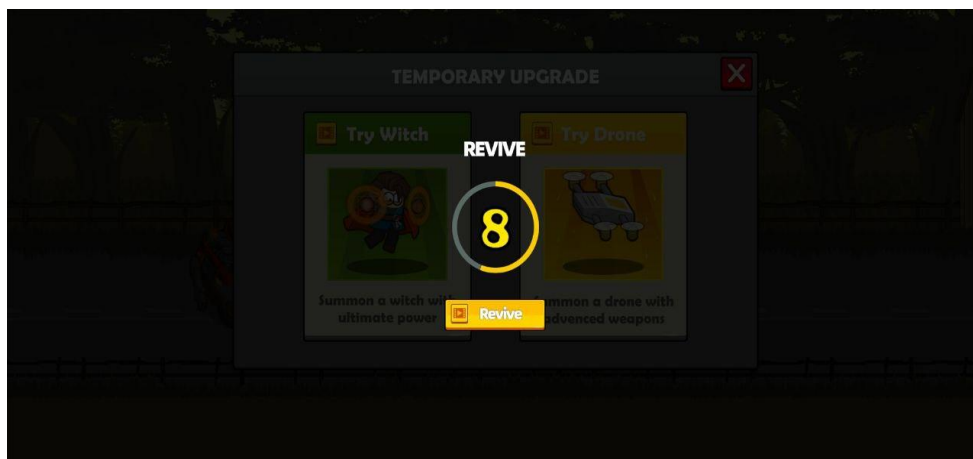


Hình 4.12. Màn hình Game Play

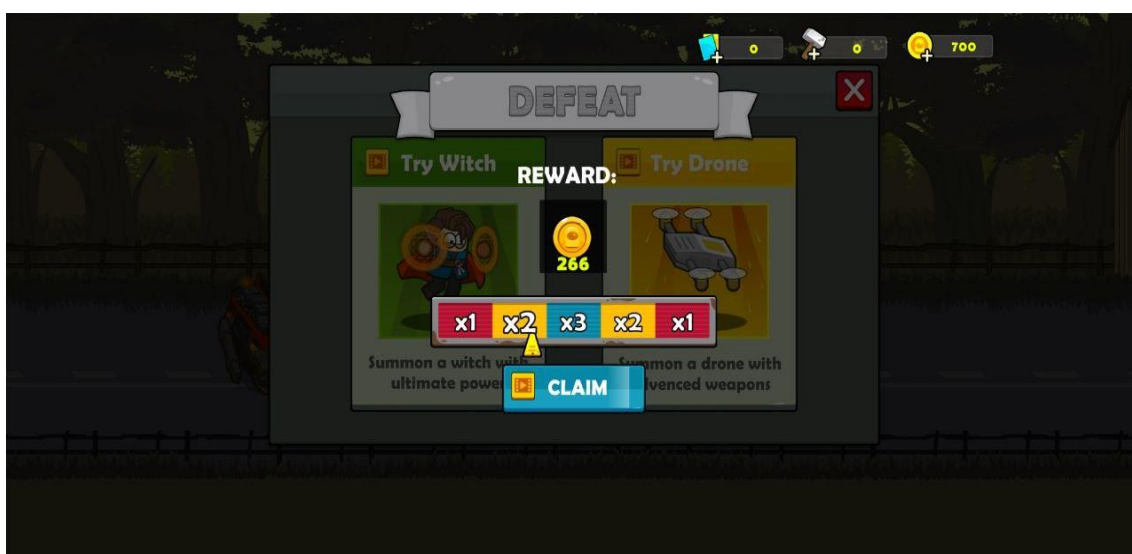
Popup chiến thắng/thất bại: Hiển thị phần thưởng, cơ hội quay thưởng.



Hình 4.13. Màn hình chiến thắng



Hình 4.14. Màn hình đếm ngược thất bại



Hình 4.15. Màn hình thất bại

4.1.7. Mục tiêu phát triển

Dự án được phát triển với mục tiêu xây dựng một trò chơi di động thể Hyper-Casual có lối chơi đơn giản nhưng hấp dẫn, vận hành mượt mà trên các thiết bị Android và có thể phát hành trên CH Play. Trò chơi hướng đến đối tượng người chơi từ 16 tuổi trở lên, yêu thích các tựa game hành động giải trí ngắn hạn, đặc biệt là thể loại game zombie, dễ tiếp cận và không đòi hỏi kỹ năng phức tạp.

Thông qua quá trình thực hiện, nhóm mong muốn áp dụng toàn diện kiến thức về lập trình C# và công cụ Unity để phát triển một sản phẩm game hoàn chỉnh từ giao diện, gameplay, logic điều khiển, cho đến hệ thống lưu trữ dữ liệu và nâng cấp. Bên cạnh đó, dự án cũng nhằm tạo nền tảng để nghiên cứu và phát triển các tính năng nâng cao hơn trong tương lai như game 3D, chế độ nhiều người chơi (multiplayer) hoặc phát hành thương mại.

Trò chơi được xây dựng trên nền tảng Android với công cụ Unity Engine, sử dụng C#, Spine Animation, GitLab và Fork để quản lý mã nguồn và phối hợp làm việc nhóm. Nội dung game bao gồm nhân vật chính và kẻ địch là zombie, các chế độ chơi đa dạng như Story, Endless, Boss và Collection, cùng hệ thống nâng cấp vũ khí, phương tiện và phần thưởng phong phú. Trò chơi cũng được thiết kế với giao diện người dùng thân thiện (UI/UX) và được đóng gói thành tệp cài đặt APK để phân phối.

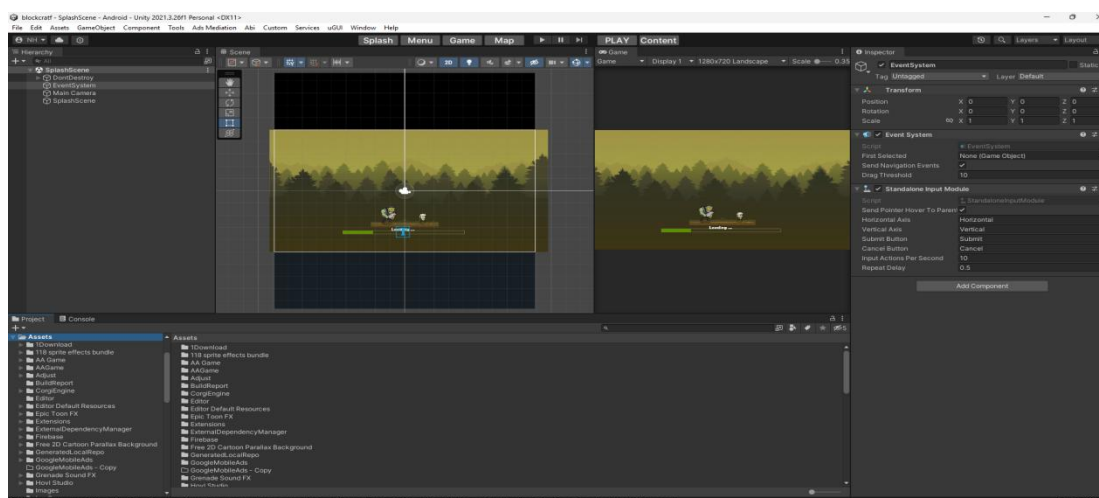
4.3.Công nghệ và công cụ sử dụng

4.3.1. Unity Engine

Unity Engine là nền tảng chính được sử dụng để phát triển trò chơi trong dự án. Đây là một công cụ phát triển game đa nền tảng mạnh mẽ, hỗ trợ cả môi trường 2D lẫn 3D, phù hợp cho cả các lập trình viên chuyên nghiệp và người mới bắt đầu. Unity cung cấp một bộ công cụ toàn diện cho việc xây dựng, tối ưu hóa và xuất bản trò chơi trên nhiều nền tảng khác nhau như Android, iOS, PC, hay các hệ máy console. Ngôn ngữ lập trình chính mà Unity hỗ trợ là C#, vốn nổi tiếng với tính dễ học, rõ ràng, nhưng vẫn đảm bảo sức mạnh và sự linh hoạt trong phát triển phần mềm.

Bên cạnh đó, Unity sở hữu Asset Store – một thư viện phong phú gồm hàng triệu tài nguyên miễn phí và trả phí, từ mô hình 3D, hình ảnh, âm thanh, hiệu ứng cho đến các plugin và công cụ hỗ trợ. Điều này giúp rút ngắn đáng kể thời gian phát triển và tối ưu quy trình làm việc. Ngoài ra, Unity còn tích hợp hệ thống Prefab và Component-Based Architecture, giúp quản lý tài nguyên và tổ chức dự án một cách khoa học, đồng thời hỗ trợ các tính năng nâng cao như vật lý thời gian thực, hệ thống ánh sáng toàn cục (Global Illumination), và công cụ debug trực quan.

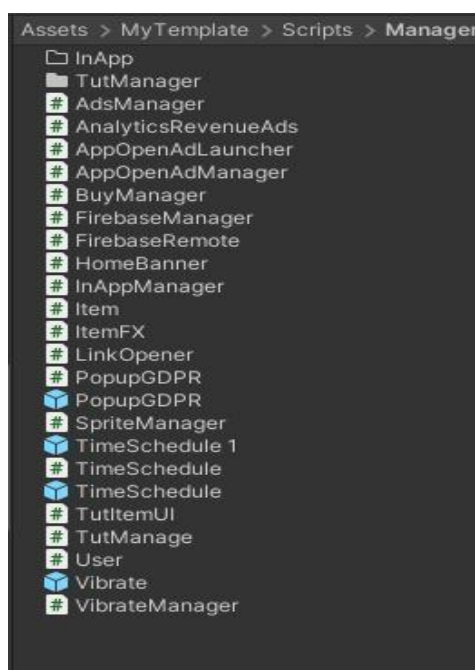
Nhờ vào sự linh hoạt, khả năng mở rộng và cộng đồng hỗ trợ lớn mạnh, Unity Engine trở thành giải pháp lý tưởng để triển khai dự án này, đồng thời tạo tiền đề cho việc nâng cấp và mở rộng sản phẩm trong tương lai.



Hình 4.16. Giao diện Unity

4.3.2. Ngôn ngữ lập trình C#

Ngôn ngữ lập trình được sử dụng để xây dựng toàn bộ logic gameplay, bao gồm các yếu tố tương tác với người dùng, hệ thống trí tuệ nhân tạo (AI) cho nhân vật zombie, hệ thống nâng cấp và quản lý level. Bên cạnh đó, các Design Pattern như Singleton, Observer và State được áp dụng nhằm tổ chức mã nguồn một cách rõ ràng, dễ bảo trì và thuận tiện cho việc mở rộng trong tương lai.

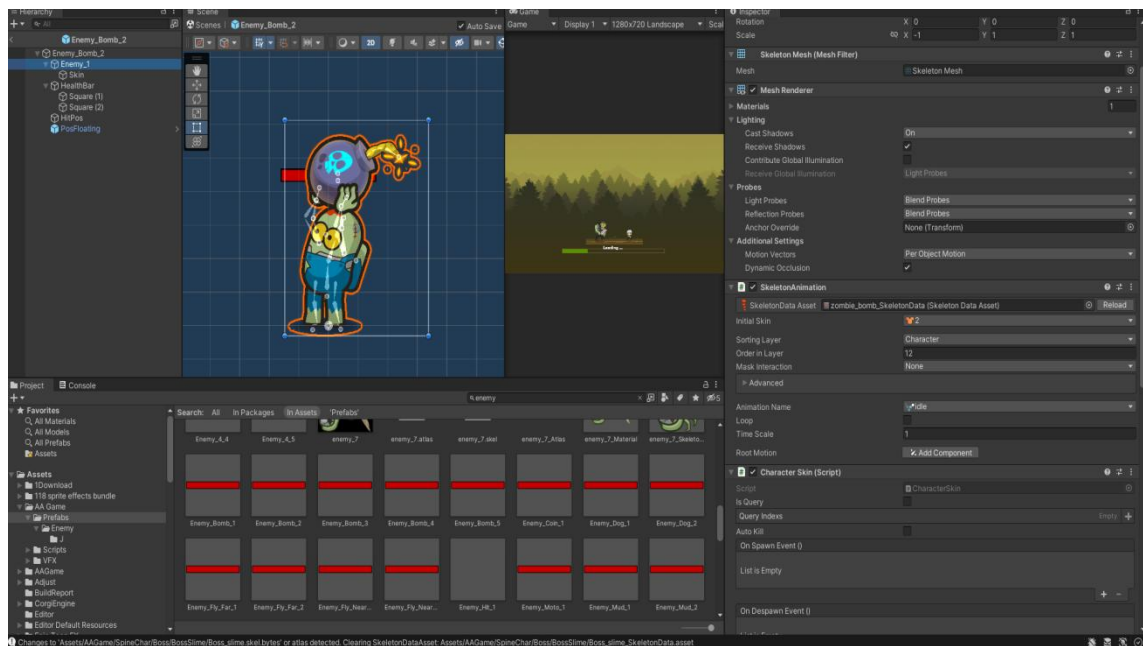


Hình 4.17. Một phần nhỏ mã C# được viết trong dự án

4.3.3. Spine Animation

Spine Animation là công cụ chuyên dụng dùng để tạo chuyển động mượt mà và tự nhiên cho các nhân vật, chẳng hạn như nhân vật chính và zombie trong

trò chơi. Thay vì sử dụng các sprite sheet truyền thống với nhiều khung hình, Spine cho phép xây dựng chuyển động dựa trên hệ thống xương (skeleton), từ đó giảm đáng kể dung lượng tài nguyên hình ảnh và tối ưu hiệu suất trò chơi. Bên cạnh đó, Spine còn mang đến khả năng tùy chỉnh linh hoạt, giúp nhà phát triển dễ dàng tạo ra các hiệu ứng hình ảnh sinh động, phù hợp với phong cách nghệ thuật của sản phẩm. Nhờ vậy, việc triển khai Spine Animation không chỉ tiết kiệm thời gian và công sức trong quá trình thiết kế, mà còn nâng cao trải nghiệm thị giác của người chơi.



Hình 4.18. Nhân vật được sử dụng Spine Animation

4.3.4. Hệ thống cửa hàng– IAP (In-App Purchasing)

Trong trò chơi, hệ thống cửa hàng trực tuyến được triển khai thông qua Unity IAP (In-App Purchasing), một giải pháp mạnh mẽ và bảo mật cao do Unity cung cấp. Công cụ này cho phép tích hợp cơ chế mua hàng trực tiếp trong ứng dụng, giúp người chơi dễ dàng sở hữu các vật phẩm cao cấp như vàng, mảnh nhân vật, gói nâng cấp đặc biệt hoặc các loại đạn mạnh mà không cần xem quảng cáo. Unity IAP hỗ trợ thanh toán qua Google Play, đảm bảo tính an toàn và thuận tiện trong quá trình giao dịch.

Việc tích hợp hệ thống IAP mang lại nhiều lợi ích quan trọng cho dự án. Trước hết, nó tạo ra nguồn doanh thu trực tiếp, đóng vai trò là một trong những kênh thương mại chính của trò chơi. Bên cạnh đó, IAP giúp nâng cao trải nghiệm

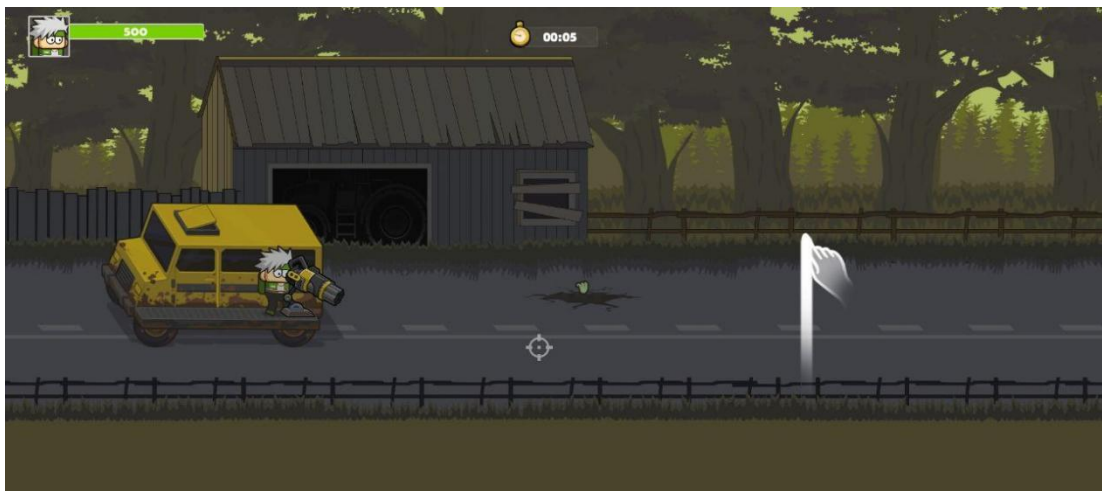
người chơi bằng cách cung cấp thêm nội dung cao cấp cho những ai có nhu cầu mở rộng và nâng cấp trải nghiệm game. Nhờ vậy, hệ thống cửa hàng không chỉ góp phần thương mại hóa sản phẩm mà còn gia tăng tính cá nhân hóa và giá trị giải trí tổng thể.

4.4. Gameplay và giao diện người dùng (UI/UX)

4.4.1. Thiết kế Gameplay

Trò chơi *Shooting Zombie* thuộc thể loại Hyper-Casual, tập trung vào sự đơn giản và dễ tiếp cận nhưng vẫn đảm bảo tính hấp dẫn và khả năng gây nghiện cho người chơi. Gameplay được thiết kế xoay quanh những cơ chế cốt lõi, giúp người chơi nhanh chóng nắm bắt và trải nghiệm ngay từ lần chơi đầu tiên.

Người chơi sẽ điều khiển khẩu súng thông qua thao tác kéo và chạm trên màn hình cảm ứng, định vị tọa độ bắn chính xác để tiêu diệt các đợt zombie liên tục xuất hiện. Trong suốt quá trình chơi, chiếc xe của nhân vật sẽ tự động di chuyển, tạo ra cảm giác nhịp độ nhanh và liên tục. Zombie sẽ xuất hiện theo từng đợt (wave) với độ khó tăng dần, buộc người chơi phải quan sát nhanh và phản xạ chính xác để bảo vệ xe khỏi bị tấn công.



Hình 4.19. Giao diện điều khiển trong game

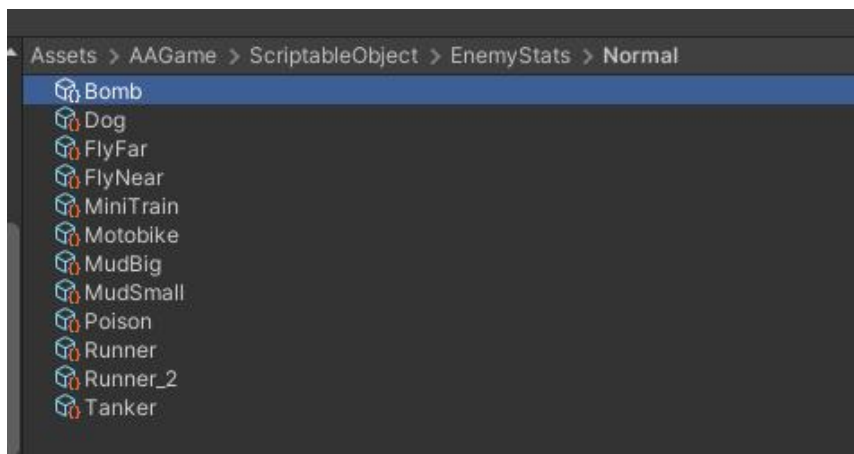
Bên cạnh cơ chế điều khiển đơn giản, trò chơi *Shooting Zombie* còn được xây dựng với nhiều trạng thái và tính năng hỗ trợ để tạo chiều sâu và giữ chân người chơi. Hệ thống trạng thái trong game bao gồm: Free – nơi người chơi có thể lựa chọn loại đạn hoặc vật phẩm hỗ trợ trước khi bắt đầu màn chơi; Playing – trạng thái chính khi gameplay diễn ra, zombie xuất hiện liên tục và tấn công xe của nhân vật; Win/Lose – trạng thái kết thúc màn chơi, hiển thị kết quả và phần

thưởng; và TutPause – trạng thái tạm dừng khi người chơi tham gia phần hướng dẫn (tutorial).

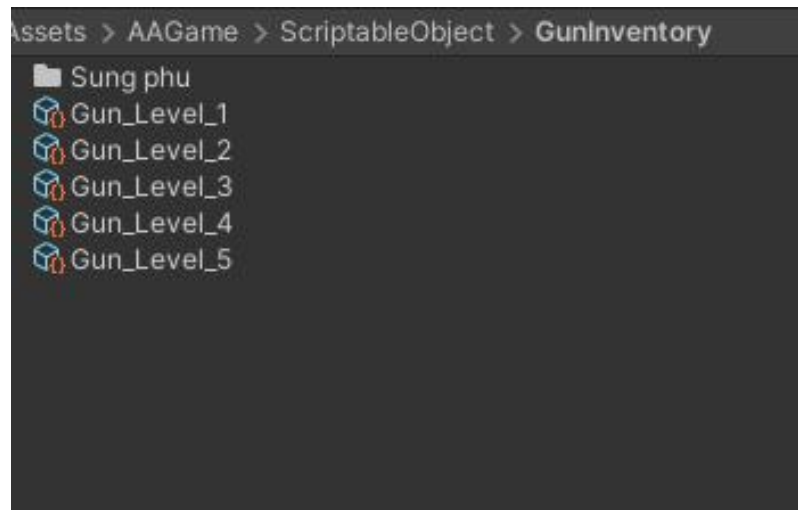
Trò chơi cũng được trang bị hệ thống nâng cấp phong phú, cho phép người chơi nâng cao sức mạnh của súng (tăng sát thương), xe (tăng chỉ số máu) và đạn (bao gồm các loại đạn mạnh, đạn nổ, đạn xuyên...). Tài nguyên để nâng cấp có thể kiếm được thông qua quá trình chơi, hoàn thành nhiệm vụ, xem quảng cáo hoặc thông qua hệ thống mua hàng trong ứng dụng (IAP).

Cơ chế tiến độ được chia theo từng cấp độ (level), mỗi màn chơi sở hữu số lượng và loại zombie riêng, giúp tăng dần độ thử thách. Đặc biệt, cứ sau mỗi năm màn chơi, một trận chiến với boss mạnh hơn sẽ xuất hiện, đòi hỏi người chơi vận dụng chiến thuật hợp lý cùng việc nâng cấp trang bị để vượt qua.

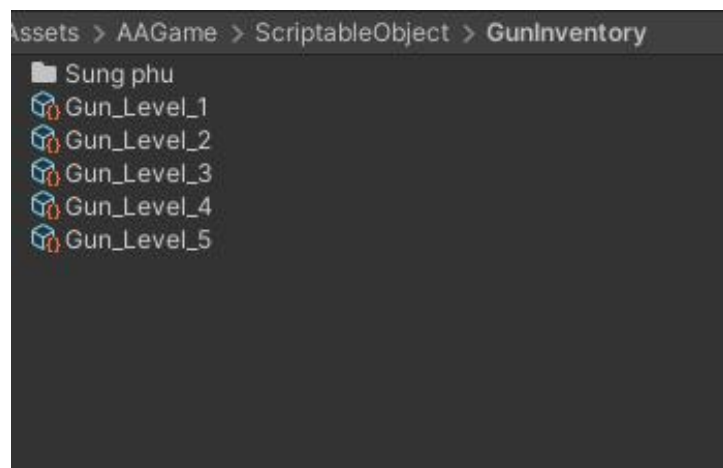
Ngoài ra, trò chơi cung cấp nhiều chế độ chơi đa dạng để tăng tính giải trí và kéo dài thời lượng trải nghiệm. *Campaign Mode* cho phép người chơi theo dõi câu chuyện và vượt qua các màn chơi tuần tự, trong khi *Boss Mode* tập trung vào việc hạ gục các boss để nhận phần thưởng cao cấp. *Endless Mode* mang đến trải nghiệm sinh tồn liên tục, thử thách phản xạ và khả năng tối ưu hóa trang bị. Cuối cùng, *Collection Mode* tạo điểm nhấn thú vị với những màn rượt đuổi zombie mang vàng, tăng cơ hội thu thập tài nguyên và vật phẩm giá trị.



Hình 4.20. Dữ liệu các Enemy trong game



Hình 4.21. Dữ liệu chỉ số súng trong Game



Hình 4.22. Dữ liệu súng theo level trong game

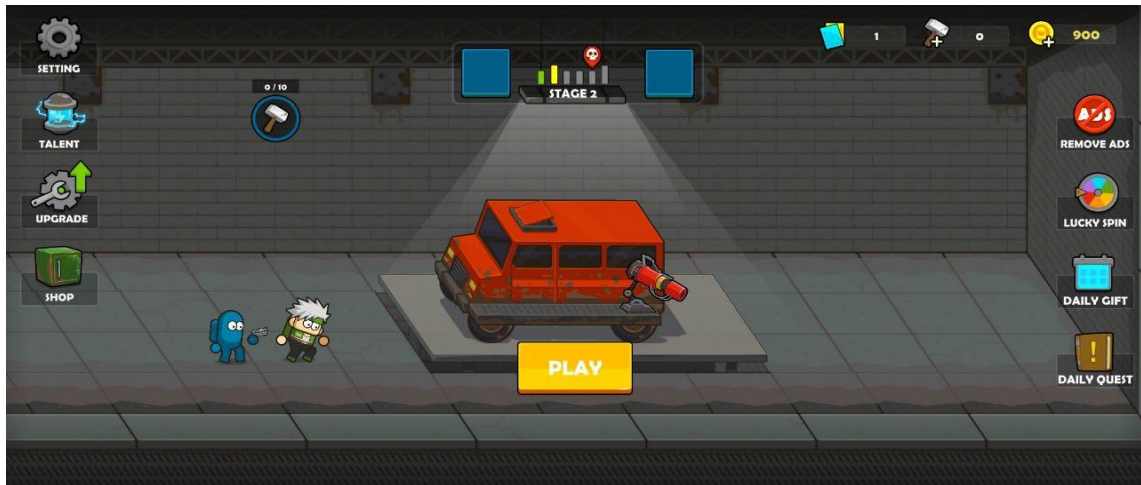
Inspector		
Fly Far (Enemy Stats Data)		
Script: EnemyStatsData		
▼ Enemy Stats Normal 30 items 1 / 2		
Hp	175	X
Atk	15	
Hp	183	X
Atk	15	
Hp	192	X
Atk	15	
Hp	201	X
Atk	15	
Hp	211	X
Atk	15	
Hp	225	X
Atk	15	
Hp	240	X
Atk	15	
Hp	256	X
Atk	15	
Hp	273	X
Atk	15	
Hp	292	X
Atk	15	
Hp	342	X
Atk	15	
Hp	392	X
Atk	15	
Hp	442	X
Atk	15	
Hp	492	X
Atk	15	
Hp	542	X
Atk	15	

Hình 4.23. Ví dụ dữ liệu minh họa trong game

4.4.2. Thiết kế giao diện người dùng (UI/UX)

Giao diện người dùng của trò chơi được thiết kế với màu sắc bắt mắt, bố cục trực quan và thao tác đơn giản, phù hợp với nhiều đối tượng người chơi. Trong đó, màn hình chính (*Main Menu*) đóng vai trò trung tâm, lấy bối cảnh một trạm sửa xe làm nền, nơi các nhân vật tương tác và chuẩn bị cho hành trình chiến đấu. Tại đây, các nút chức năng được bố trí rõ ràng và dễ nhận biết, bao gồm: Fight – bắt đầu màn chơi; Shop – truy cập cửa hàng để mua vật phẩm; Upgrade – thực hiện nâng cấp xe và vũ khí; cùng các mục Quest, Daily Login, Lucky Spin,

và Talent giúp người chơi tham gia nhiệm vụ, đăng nhập nhận thưởng, quay may mắn, và phát triển kỹ năng nhân vật. Cách bố trí giao diện này không chỉ tối ưu hóa trải nghiệm người dùng mà còn tạo sự thuận tiện, giúp người chơi dễ dàng làm quen và thao tác ngay từ lần đầu trải nghiệm.

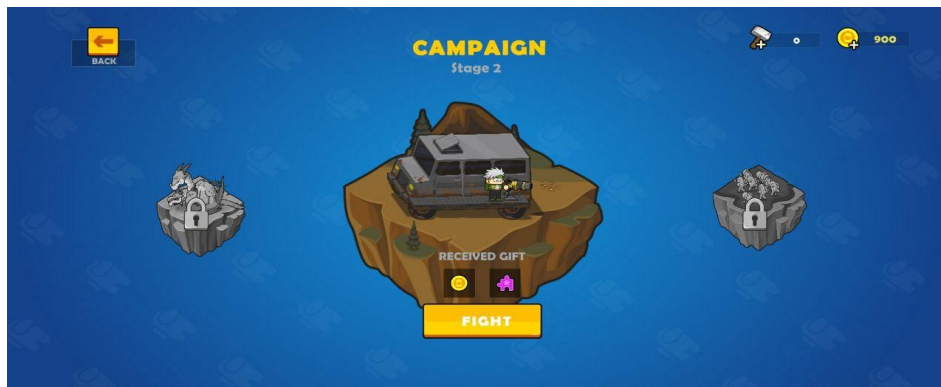


Hình 4.24. Màn hình bối cảnh chính

Màn hình chọn chế độ (Mode Select):

Cho phép chọn các chế độ Campaign, Boss, Endless...

Hiển thị phần thưởng tương ứng cho mỗi chế độ.



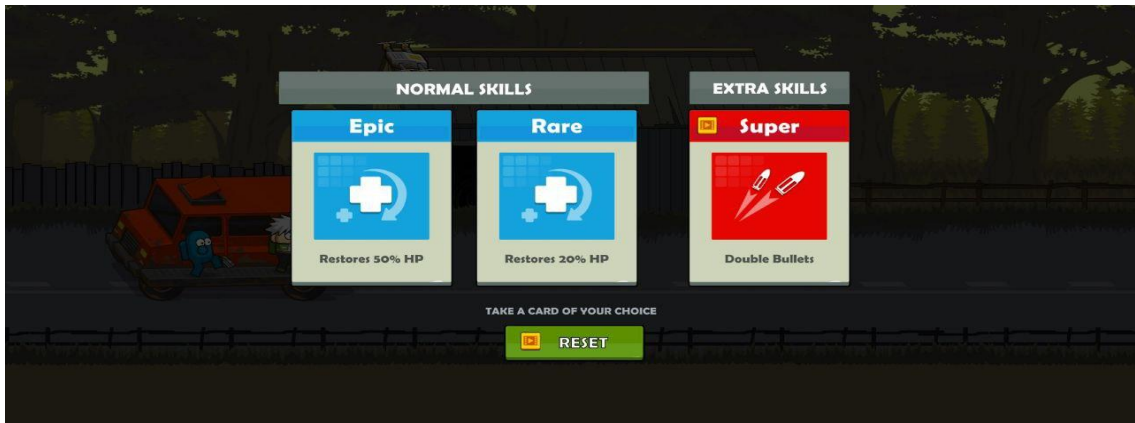
Hình 4.25. Màn hình chọn chế độ

Màn hình gameplay:



Hình 4.26. Màn hình Game Play

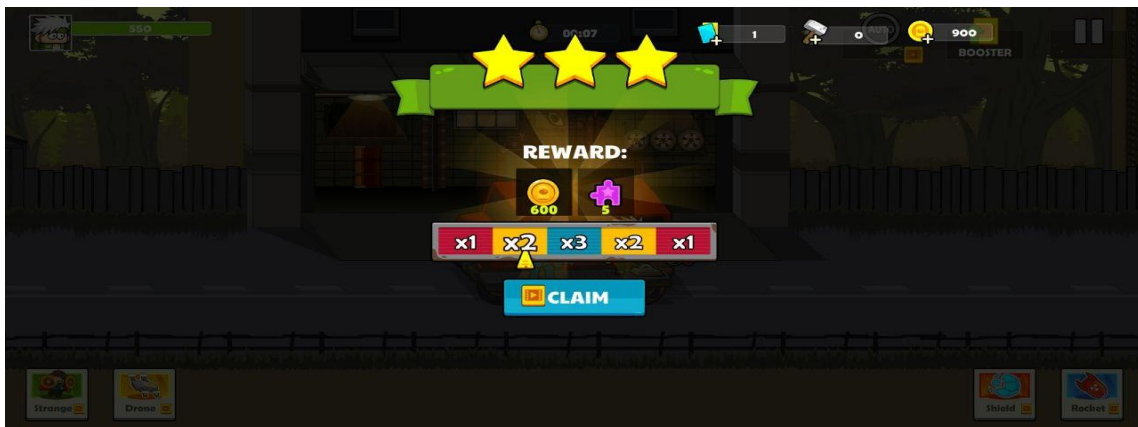
Popup hiện hỗ trợ như xem quảng cáo nhận buff, hoặc chọn nâng cấp khi qua wave.



Hình 4.27. Hệ thống nâng cấp khi qua wave mới

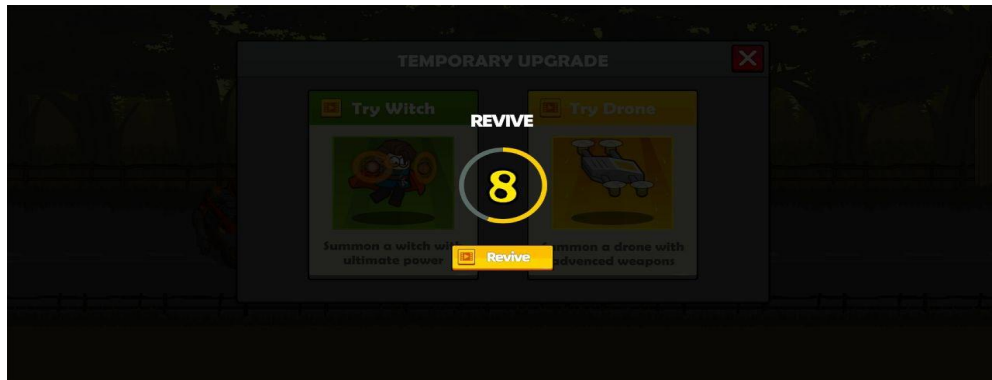
Màn hình chiến thắng/thất bại:

- Thắng: hiện số lượng tài nguyên nhận được, phần thưởng thêm nếu xem quảng cáo.

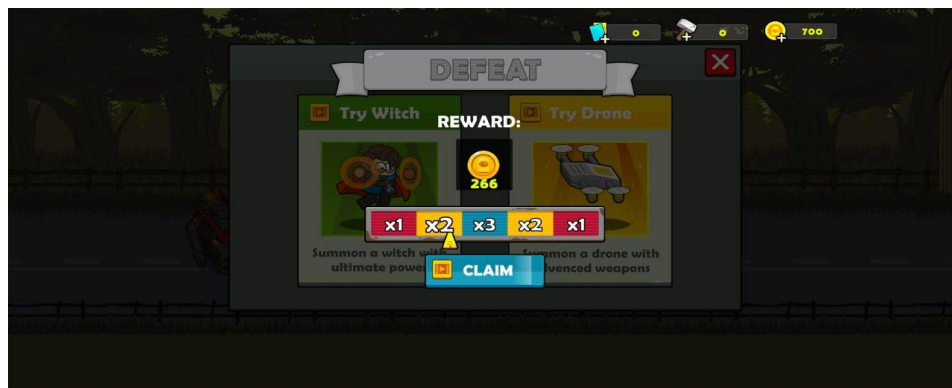


Hình 4.28. Màn hình chiến thắng game

- Thua: Hiển thị màn hình đếm ngược để tiếp tục nếu màn hình chạy hết thì hiển thị popup thua



Hình 4.29. Hình đếm ngược khi thua



Hình 4.30. Màn hình thua game

4.4.3. Nguyên tắc thiết kế UX được áp dụng

Trong quá trình thiết kế trải nghiệm người dùng (UX), trò chơi áp dụng các nguyên tắc cơ bản nhằm mang lại sự thuận tiện và hấp dẫn tối đa cho người chơi. Trước hết, trò chơi được tối giản hóa thao tác điều khiển, cho phép người dùng dễ dàng thao tác chỉ với một tay thông qua các cử chỉ đơn giản như chạm hoặc kéo. Mọi hành động trong game đều có phản hồi nhanh chóng và rõ ràng nhờ hiệu ứng âm thanh, hoạt ảnh (animation) và các cửa sổ thông báo (popup) xuất hiện tức thời, tạo cảm giác mượt mà và sống động. Ngoài ra, thiết kế còn chú trọng đến yếu tố kích thích người chơi thông qua hệ thống phần thưởng phong phú, chế độ nhân đôi tài nguyên, vòng quay may mắn và nhiều yếu tố khích lệ khác. Để giữ chân người chơi lâu dài, game liên tục mang đến các nhiệm vụ hàng ngày, hệ thống nâng cấp đa dạng và một giao diện trực quan, dễ sử dụng, đảm bảo trải nghiệm mượt mà và tạo động lực khám phá lâu dài.

4.5. Hệ thống nhiệm vụ và phần thưởng

Hệ thống nhiệm vụ trong trò chơi đa dạng, được phân loại nhằm mang lại trải nghiệm phong phú và kích thích tính cạnh tranh, bao gồm:

Nhiệm vụ hằng ngày (Daily Quest): Đây là nhóm nhiệm vụ được làm mới mỗi 24 giờ, yêu cầu người chơi thực hiện các hoạt động cơ bản như đăng nhập hằng ngày, tiêu diệt một số lượng zombie nhất định, xem quảng cáo hoặc hoàn thành một số trận đấu. Phần thưởng cho các nhiệm vụ này thường là vàng, vật phẩm nâng cấp, lượt quay may mắn hoặc các chỉ số tạm thời, giúp người chơi duy trì hứng thú chơi game mỗi ngày.

Nhiệm vụ theo tiến độ (Progress Quest): Loại nhiệm vụ này gắn liền với những cột mốc quan trọng trong game như mở khóa chế độ chơi mới, nâng cấp vũ khí đạt cấp độ nhất định hoặc đánh bại các boss quan trọng. Phần thưởng của nhóm nhiệm vụ này có giá trị cao hơn, đôi khi còn giúp mở khóa thêm tính năng mới hoặc vật phẩm hiếm, qua đó tạo động lực để người chơi tiến xa hơn.

Nhiệm vụ phụ (Side Quest): Các nhiệm vụ này xuất hiện ngẫu nhiên trong quá trình chơi, chẳng hạn như yêu cầu tiêu diệt zombie bằng loại đạn đặc biệt hoặc vượt qua một đợt tấn công mà không mất máu. Việc bổ sung các nhiệm vụ ngẫu nhiên này làm tăng tính bất ngờ, đa dạng hóa trải nghiệm và tạo thêm thử thách thú vị cho người chơi.

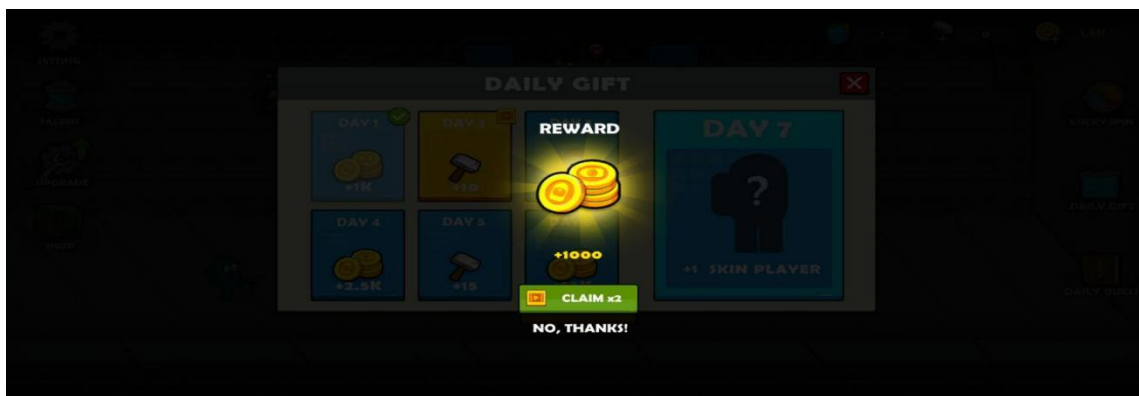
Nhiệm vụ đặc biệt (Special/Boss Quest): Đây là nhóm nhiệm vụ liên quan đến các chế độ đặc biệt như Boss Mode hoặc các sự kiện giới hạn theo thời gian. Khi hoàn thành, người chơi sẽ nhận được phần thưởng độc quyền, tạo cảm giác thỏa mãn và nâng cao giá trị sưu tầm trong trò chơi.



Hình 4.31. Popup hệ thống nhiệm vụ

4.5.3. Hệ thống phần thưởng

Sau mỗi trận đấu, người chơi sẽ được hiển thị một cửa sổ popup phần thưởng. Khi giành chiến thắng, người chơi sẽ nhận được vàng và các mảnh vật phẩm. Ngoài ra, hệ thống còn cung cấp tùy chọn nhận đôi phần thưởng bằng cách xem quảng cáo thông qua IronSource.



Hình 4.32. Popup nhận thưởng



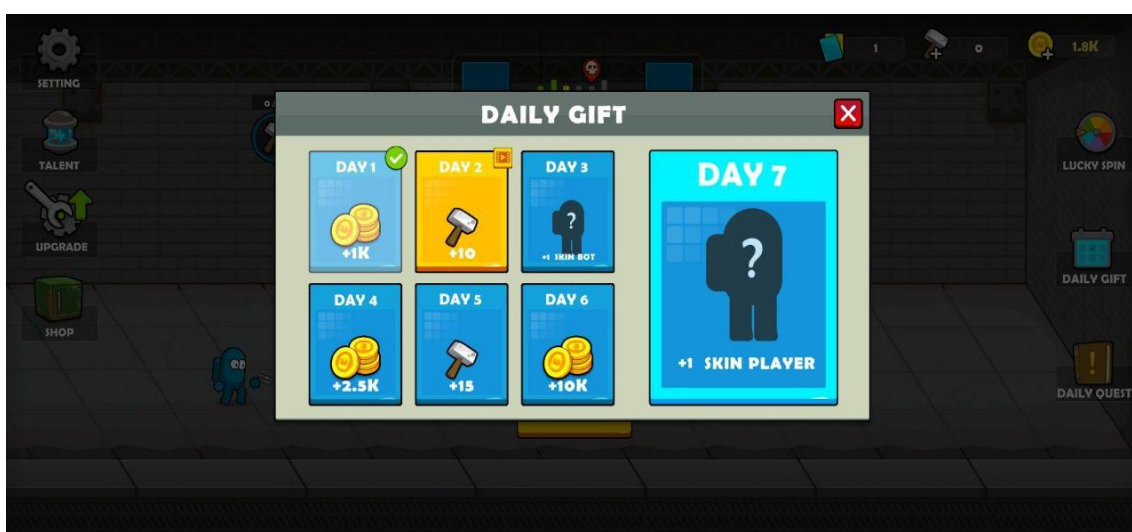
Hình 4.33. Popup phần thưởng mỗi trận

Người chơi mỗi ngày sẽ có cơ hội tham gia vòng quay may mắn để nhận phần thưởng hấp dẫn. Ngoài ra, người chơi cũng có thể nhận thêm lượt quay khi xem quảng cáo trong trò chơi.



Hình 4.34. Popup Lucky Spin

Hộp quà đăng nhập (Daily Login Gift) là hệ thống phần thưởng kéo dài trong 7 ngày, trong đó giá trị quà tặng sẽ tăng dần theo số ngày người chơi đăng nhập liên tiếp.



Hình 4.35. Popup nhận quà hằng ngày

4.5.4. Động lực hóa người chơi

Động lực hóa người chơi được thực hiện thông qua hệ thống nhiệm vụ kết hợp với các phần thưởng hấp dẫn, giúp mang lại cho người chơi cảm giác có mục tiêu rõ ràng trong mỗi phiên chơi. Nhờ đó, trò chơi khuyến khích người chơi quay lại thường xuyên hơn, nâng cao tỷ lệ duy trì và mức độ gắn bó. Đồng thời, hệ thống này còn khéo léo kích thích người chơi tương tác với quảng cáo hoặc

các gói mua hàng trong ứng dụng (IAP) một cách tự nguyện, tạo nên trải nghiệm giải trí liền mạch và gia tăng giá trị cho cả người chơi lẫn nhà phát triển.

4.6. Hệ thống điều khiển và Enemy(AI)

4.6.1. Hệ thống điều khiển nhân vật người chơi

Trò chơi được xây dựng với cơ chế điều khiển một chạm, tối ưu hóa cho thiết bị di động nhằm đảm bảo thao tác đơn giản nhưng vẫn tạo được sức hút đối với người chơi. Người chơi chỉ cần kéo hoặc thả ngón tay để điều chỉnh vị trí tâm súng, trong khi hệ thống tự động khai hỏa khi mục tiêu xuất hiện trong phạm vi tấn công. Cơ chế bắn được thiết kế đa dạng, trong đó mỗi loại đạn sở hữu hiệu ứng riêng như gây nổ lan, xuyên mục tiêu hay tạo hiệu ứng lửa, góp phần tăng tính chiến thuật và mức độ hấp dẫn trong quá trình chiến đấu.

Bên cạnh đó, hệ thống vũ khí cho phép nâng cấp các thuộc tính như tốc độ bắn, sát thương và hiệu ứng đặc biệt, giúp người chơi cảm nhận rõ ràng sự gia tăng sức mạnh qua từng giai đoạn. Nhân vật điều khiển (xe) được thiết kế tự động di chuyển, giúp người chơi tập trung vào việc ngắm bắn và xử lý tình huống, qua đó duy trì nhịp độ nhanh và tạo cảm giác cuốn hút đặc trưng của thể loại game Hyper-Casual.

4.6.2. Hệ thống AI của zombie

Zombie là kẻ thù chủ yếu trong trò chơi, được thiết kế với đa dạng loại hình và trí tuệ nhân tạo nhằm gia tăng tính hấp dẫn và thử thách cho người chơi. Về hành vi cơ bản, zombie xuất hiện liên tục theo từng đợt (wave) và tự động di chuyển về phía xe của người chơi; khi tiếp xúc với xe, chúng gây sát thương nếu chưa bị tiêu diệt.

Ngoài ra, một số zombie sở hữu AI nâng cao, thể hiện qua các đặc tính riêng biệt. Ví dụ, zombie chạy nhanh ưu tiên tiếp cận người chơi, zombie tấn công từ xa giữ khoảng cách để gây sát thương, zombie bay có khả năng né tránh một số loại đạn, zombie nổ phát nổ khi đến gần xe, và các boss với nhiều giai đoạn, khả năng gọi minion hỗ trợ cùng mức phòng thủ cao.

Cấu trúc AI cơ bản của các zombie được quản lý thông qua mô hình Finite State Machine (FSM), với các trạng thái tuần tự gồm: Idle → MoveToTarget → Attack → Die. Thiết kế này cho phép dễ dàng mở rộng và tùy biến hành vi cho

từng loại zombie khác nhau, tạo ra sự đa dạng trong trải nghiệm chiến đấu của người chơi.

4.6.3. Tối ưu hiệu suất AI

Để đảm bảo hiệu năng trong quá trình chơi, các zombie được triển khai theo cơ chế object pool, giúp tránh việc tạo và xóa đối tượng liên tục gây tốn tài nguyên. AI của zombie chỉ được kích hoạt khi chúng nằm trong vùng quan sát của camera (camera bounds), nhờ đó giảm thiểu tính toán không cần thiết. Bên cạnh đó, LOD (Level of Detail) logic được áp dụng: zombie ở khoảng cách xa sẽ có hành vi đơn giản hơn, góp phần tiết kiệm hiệu năng hệ thống.

4.6.4. Tính năng đặc biệt

AI zombie kết hợp với cơ chế vật lý, tạo ra các phản ứng linh hoạt như bị đẩy lùi, nổ văng, vỡ mảnh hoặc hiệu ứng ragdoll nhẹ. Các hành vi này có thể được tùy biến theo từng chế độ chơi, từ đó mang lại trải nghiệm đa dạng và mới mẻ cho người chơi.

4.7. Hệ thống nâng cấp

4.7.1. Mục tiêu của hệ thống

Hệ thống nâng cấp được thiết kế nhằm tạo động lực để người chơi tiếp tục tham gia trò chơi và tích lũy tài nguyên. Nó cho phép người chơi cá nhân hóa lối chơi thông qua việc nâng cấp theo chiến lược riêng, đồng thời tăng dần độ khó theo tiến trình, buộc người chơi phải cải thiện khả năng và trang bị để vượt qua các màn chơi tiếp theo.

4.7.2. Các loại nâng cấp trong game

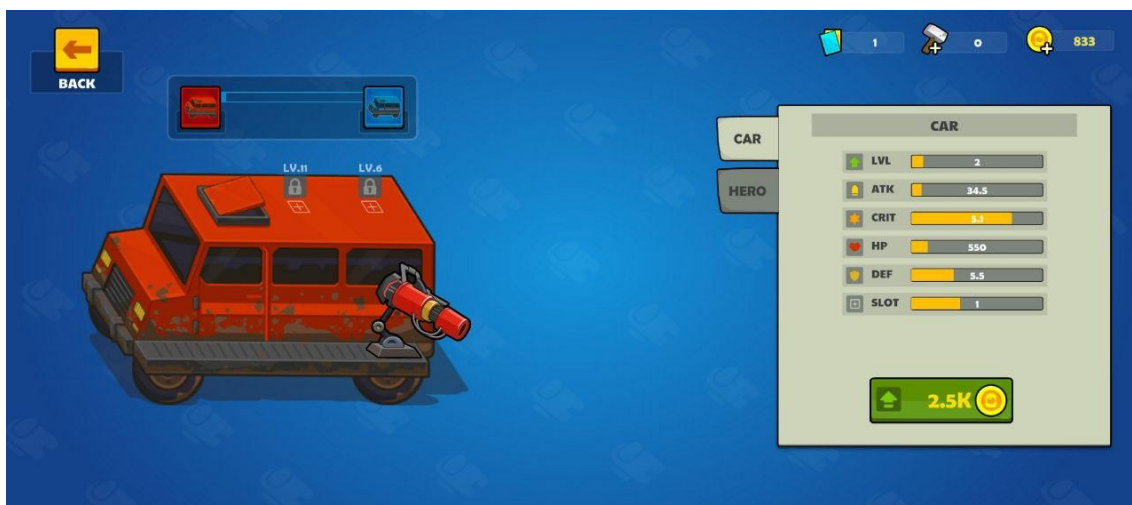
Hệ thống nâng cấp xe cho phép cải thiện các thuộc tính cơ bản nhằm gia tăng khả năng sinh tồn và hiệu quả chiến đấu:

Tăng lượng máu của xe: Giúp xe trụ vững lâu hơn trước các đợt tấn công của zombie.

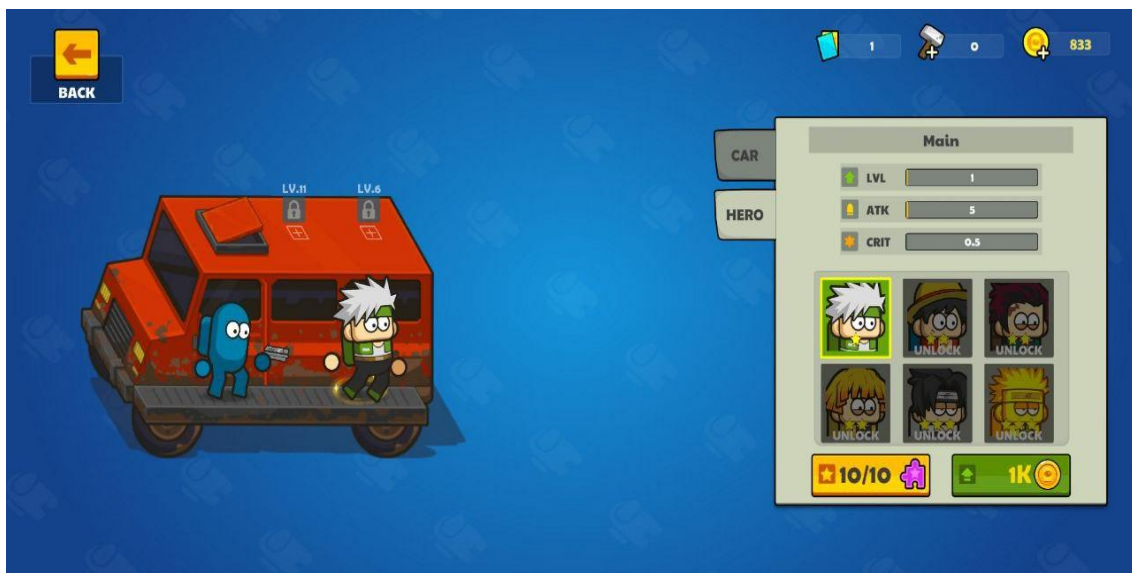
Tăng tốc độ xe: Áp dụng trong một số chế độ chơi, giúp người chơi cơ động hơn trên bản đồ.

Nâng cấp độ bền: Giảm sát thương nhận vào, tăng khả năng chống chịu trước các đòn tấn công.

Tăng số lượng nhân vật trong xe: Cho phép vận chuyển thêm đồng đội, mở rộng khả năng chiến lược và hỗ trợ trong trận đấu.



Hình 4.36. Màn hình nâng cấp xe



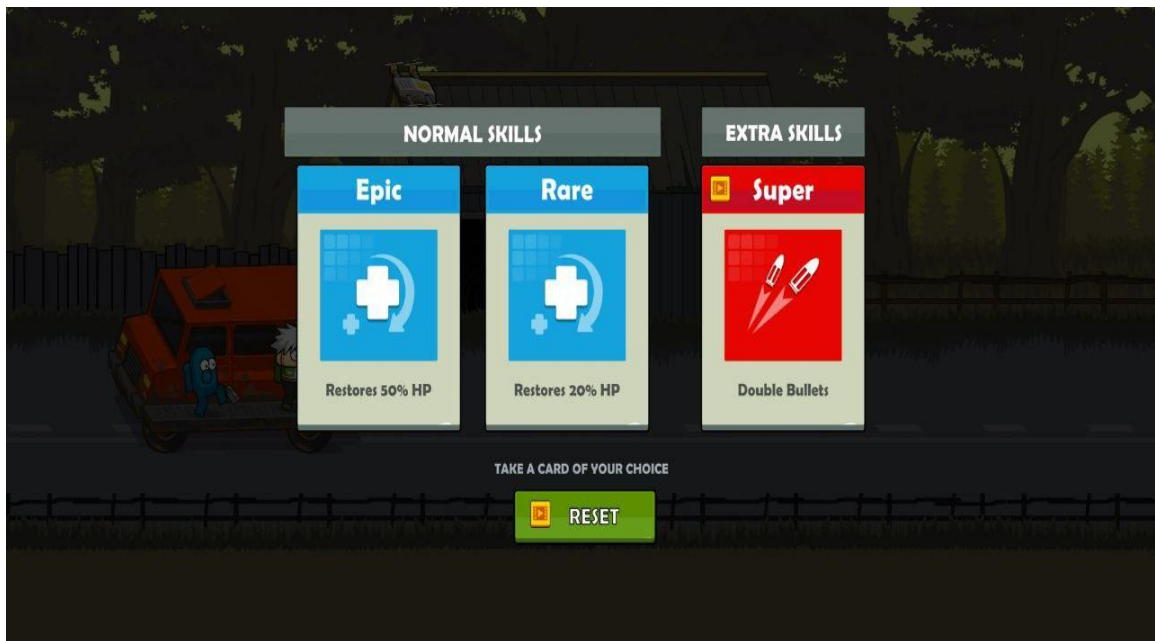
Hình 4.37. Màn hình nâng cấp nhân vật trên xe

Hệ thống nâng cấp đạn cho phép người chơi cải thiện loại đạn và hiệu ứng của chúng, từ đó tăng khả năng chiến đấu:

Các loại đạn: Từ đạn thường, tiến tới đạn lửa, đạn băng, đạn nổ và các loại đặc biệt khác.

Hiệu ứng riêng biệt: Mỗi loại đạn mang hiệu ứng chiến thuật khác nhau, như làm chậm kẻ địch, gây nổ lan, xuyên hàng, v.v.

Tùy chọn đạn trước trận đấu: Người chơi có thể lựa chọn loại đạn sử dụng trước mỗi trận đấu, đặc biệt trong chế độ Free, để tối ưu chiến lược.



Hình 4.38. Màn hình nâng cấp đạn

4.7.3. Giao diện nâng cấp (Upgrade UI)

Giao diện nâng cấp được bố trí ngay tại màn hình chính (Main Menu) với các tab riêng biệt, bao gồm Upgrade Hero và Upgrade Car. Mỗi mục nâng cấp hiển thị rõ ràng các thông tin cần thiết như mức nâng cấp hiện tại so với mức tối đa, chi phí nâng cấp (bằng vàng hoặc mảnh), và tác dụng cụ thể của nâng cấp đó. Người chơi có thể thu thập tài nguyên để nâng cấp thông qua nhiều hình thức khác nhau, bao gồm chiến thắng các màn chơi, hoàn thành nhiệm vụ, xem quảng cáo, hoặc nạp tiền thông qua các giao dịch trong ứng dụng (IAP).

4.7.4. Hệ thống cửa hàng (Shop)

Cửa hàng trong trò chơi cung cấp nhiều loại hàng hóa và hình thức thanh toán để người chơi nâng cấp và mua sắm tiện lợi:

Loại hàng hóa: Bao gồm các gói vàng với giá trị khác nhau, gói búa nâng cấp nhà ga, và các gói Free cho phép nhận tài nguyên thông qua việc xem quảng cáo.

Hình thức thanh toán: Người chơi có thể sử dụng tài nguyên trong game như vàng hoặc mảnh, thực hiện In-App Purchase (IAP) qua Google Play, hoặc xem quảng cáo hàng ngày để nhận phần thưởng.

Giao diện Shop: Được tổ chức theo các tab phân loại như Gold, Item, Special Pack, đồng thời có các popup giới thiệu các ưu đãi đặc biệt hoặc các sự kiện (event shop) nhằm thu hút người chơi.



Hình 4.39. Hình ảnh hệ thống shop

4.7.5. Cơ chế kích thích người chơi nâng cấp

Hệ thống nâng cấp còn được thiết kế để tạo động lực và thưởng cho người chơi: mức độ khó của các màn chơi sẽ tăng dần theo thời gian, buộc người chơi phải nâng cấp để có thể vượt qua. Sau khi nâng cấp, người chơi sẽ nhận được các phần thưởng hấp dẫn, ví dụ như đạt súng cấp 5 sẽ được tặng đạn đặc biệt. Bên cạnh đó, hệ thống tích hợp quảng cáo cho phép người chơi nâng cấp miễn phí sau khi xem video, tạo cơ hội gia tăng tài nguyên mà không cần chi tiền.

4.8. Quản lý dữ liệu người chơi và lưu trữ

4.8.1. Mục tiêu của hệ thống lưu trữ dữ liệu

Hệ thống dữ liệu trong game được thiết kế để lưu trữ toàn bộ tiến trình chơi của người chơi, bao gồm các thông tin như level đã hoàn thành, số vàng, loại đạn và vật phẩm sở hữu, trạng thái nâng cấp của xe, súng, đạn, các nhiệm vụ đã hoàn thành, cũng như các chế độ chơi đã mở khóa. Thiết kế này giúp người chơi có thể quay lại trò chơi và tiếp tục từ vị trí trước đó, đồng thời cung cấp dữ liệu phục vụ việc phân tích hành vi người chơi để điều chỉnh gameplay cho phù hợp. Ngoài ra, việc lưu trữ đầy đủ còn tránh tình trạng mất dữ liệu, bảo đảm trải nghiệm chơi mượt mà và liền mạch.

4.8.2. Phương pháp lưu dữ liệu sử dụng trong game

Hệ thống lưu trữ dữ liệu trong game được triển khai theo nhiều phương thức tùy theo tính chất và mức độ quan trọng của dữ liệu.

PlayerPrefs được sử dụng để lưu trữ các dữ liệu đơn giản như âm lượng, lần chơi gần nhất hay các toggle tùy chọn. Phương thức này dễ triển khai và phù hợp cho những cấu hình không quá quan trọng.

JSON kết hợp Local Save cho phép lưu trữ dữ liệu người chơi phức tạp hơn thông qua các lớp quản lý như DataManager và SaveLoadSystem, dưới dạng các file JSON trên thiết bị. Ưu điểm của phương pháp này là dễ mở rộng, có thể mã hóa khi cần, hỗ trợ debug và test dữ liệu. Ví dụ, các file dữ liệu có thể bao gồm playerdata.json, questdata.json.

ScriptableObject được sử dụng cho các dữ liệu tĩnh như danh sách enemy, loại đạn, hay cấu hình level. Phương pháp này thuận tiện cho thao tác trực quan trong Unity Editor, giúp việc quản lý và chỉnh sửa dữ liệu trở nên dễ dàng.



Hình 4.40. Hệ thống ScriptableObject trong game

4.8.3. Cơ chế tự động lưu / load dữ liệu

Khi người chơi khởi động game, hệ thống sẽ tự động load dữ liệu từ các file JSON để khôi phục tiến trình chơi trước đó. Trong quá trình chơi, mỗi khi hoàn thành nhiệm vụ, chiến thắng trận đấu hoặc thực hiện nâng cấp, dữ liệu sẽ được tự động lưu (auto-save), đảm bảo các thay đổi được cập nhật kịp thời. Nhờ cơ chế này, ngay cả khi người chơi thoát game hoặc máy tắt đột ngột, dữ liệu vẫn được bảo toàn, miễn là auto-save đã được thực hiện đúng thời điểm.

```

02 references
2 > public class User : ScriptableObject ...
5
7 [System.Serializable]
4 references
3 > public class UserData ...
3
4 [System.Serializable]
5 references
5 > public class ItemValue ...
7
8 [System.Serializable]
3 references
9 > public class ItemValueFloat ...
3
9 [System.Serializable]
3 references
9 > public class Player ...
5
5 [System.Serializable]
0 references
7 > public class Car ...
3
1 reference
4 > public enum CarName ...
3
9 [System.Serializable]
5 references
9 > public class StarLevel ...
5
5
7
8 //=====USER INVENTORY=====//

```

Hình 4.41. Hệ thống lưu game

CHƯƠNG 5: KIỂM THỬ VÀ TỐI ƯU HIỆU NĂNG

5.1. Mục tiêu kiểm thử

Quá trình kiểm thử được thực hiện nhằm đảm bảo rằng trò chơi vận hành ổn định, không xảy ra hiện tượng crash hay treo máy. Đồng thời, các chức năng phải hoạt động đúng theo thiết kế, hiệu suất mượt mà, tránh giật lag trong quá trình chơi. Giao diện cần được kiểm tra kỹ lưỡng để đảm bảo rõ ràng, không gặp lỗi font hay sai hiển thị. Ngoài ra, trải nghiệm người dùng phải được duy trì nhất quán trên nhiều thiết bị Android khác nhau, giúp đảm bảo chất lượng chơi game đồng đều cho tất cả người chơi.

5.1.2. Các hình thức kiểm thử được áp dụng

Kiểm thử chức năng (Functional Testing): Quá trình kiểm thử chức năng tập trung vào việc đảm bảo các tính năng chính hoạt động đúng theo thiết kế. Điều này bao gồm nâng cấp súng, xe, đạn; giao diện shop, nhiệm vụ và popup; trải nghiệm chơi các chế độ như Story, Boss, Endless; cũng như hoạt động của quảng cáo IronSource và IAP. Đồng thời, kiểm thử phải đảm bảo logic xử lý dữ liệu chính xác, không xảy ra sai lệch.

Kiểm thử giao diện (UI/UX Testing): Kiểm thử giao diện nhằm đảm bảo giao diện không bị vỡ hoặc lệch bố cục trên nhiều độ phân giải khác nhau, đồng thời kiểm tra phản hồi của các nút bấm, thanh điều hướng và hiệu ứng tương tác, để trải nghiệm người dùng mượt mà và trực quan.

Kiểm thử thiết bị thật (Real Device Testing): Trò chơi được kiểm thử trên nhiều dòng máy Android, từ tầm thấp đến cao cấp. Các tiêu chí bao gồm thời gian load không vượt quá 3 giây, chạy mượt ở 60 FPS trên thiết bị tầm trung, và mức tiêu thụ CPU/RAM không vượt quá giới hạn cho phép.

Kiểm thử quảng cáo và thanh toán: Quy trình kiểm thử bao gồm xem quảng cáo để nhận phần thưởng đúng quy định, và kiểm thử IAP bằng tài khoản test Google Play để đảm bảo các giao dịch giả lập thành công.

Kiểm thử lưu trữ: Đảm bảo dữ liệu người chơi được lưu chính xác sau mỗi trận, đồng thời kiểm tra khả năng bảo toàn dữ liệu khi thoát game, mất mạng hoặc tắt máy đột ngột, tránh tình trạng mất dữ liệu ảnh hưởng trải nghiệm.

5.1.3. Tối ưu hiệu suất

Để đảm bảo trò chơi vận hành mượt mà trên các thiết bị di động, nhóm phát triển đã thực hiện nhiều biện pháp tối ưu hóa ở các khía cạnh khác nhau. Về hình ảnh, trò chơi sử dụng các định dạng nén chất lượng cao như PNG và WEBP, đồng thời áp dụng Sprite Atlas để gom các ảnh nhỏ lại, từ đó giảm số lượng draw call và nâng cao hiệu năng hiển thị.

Về hiệu ứng, thay vì sử dụng các animation nặng, nhóm đã sử dụng DOTween để tạo chuyển động mượt mà và nhẹ nhàng hơn, đồng thời giảm số lượng particle khi không thực sự cần thiết nhằm tiết kiệm tài nguyên.

Trong tối ưu script, Object Pooling được áp dụng cho enemy và đạn nhằm tránh việc Instantiate liên tục, giảm tải cho bộ nhớ và CPU. Logic update cũng được tối ưu bằng cách hạn chế sử dụng Update(), thay thế bằng Invoke, Coroutine hoặc Event để chỉ thực hiện các hành vi khi thật sự cần thiết.

Về âm thanh, các file được nén ở dạng mono và giảm bitrate, đồng thời chỉ phát khi cần thiết, tránh để nhiều audio chạy nền gây ảnh hưởng đến hiệu suất tổng thể của trò chơi.

5.1.4. Kết quả kiểm thử

Kết quả kiểm thử cho thấy trò chơi hoạt động ổn định trên hơn 90% các thiết bị Android được thử nghiệm. Hệ thống quảng cáo và IAP hoạt động đúng theo logic thiết kế, đảm bảo trải nghiệm người chơi liền mạch. Trong suốt quá trình kiểm thử, không phát hiện sự cố crash hay lỗi nghiêm trọng nào, chứng tỏ chất lượng và độ ổn định của trò chơi đã được đảm bảo.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Qua quá trình thực hiện khóa luận với đề tài “Thiết kế và phát triển game Shooting Zombie trên nền tảng Unity”, em đã đạt được những kết quả phù hợp với mục tiêu đặt ra. Cụ thể, em đã xây dựng được một trò chơi 2D thuộc thể loại Hyper-Casual, có lối chơi rõ ràng, đồ họa phù hợp và đáp ứng được trải nghiệm của người dùng trên thiết bị di động.

Trong quá trình phát triển, em đã vận dụng các kiến thức về lập trình hướng đối tượng với C#, sử dụng Unity Engine, thiết kế UI/UX, xây dựng hệ thống AI cho đối tượng địch và tổ chức các cơ chế gameplay. Đồng thời, em cũng áp dụng quy trình quản lý dự án với GitLab và phần mềm Fork nhằm đảm bảo tính hệ thống và khả năng mở rộng của mã nguồn.

Ngoài ra, em đã triển khai thành công các hệ thống chức năng quan trọng như nâng cấp vũ khí, xe và đạn; tích hợp quảng cáo thông qua IronSource; xây dựng hệ thống mua hàng trong ứng dụng (IAP) và lưu trữ dữ liệu người chơi. Quá trình kiểm thử và tối ưu hiệu suất đã giúp trò chơi hoạt động ổn định trên đa số thiết bị Android. Sản phẩm hiện đã được build dưới dạng tệp APK và sẵn sàng cho giai đoạn phát hành thử nghiệm nội bộ.

2. Hướng phát triển trong tương lai

Trong giai đoạn phát triển tiếp theo, em dự định mở rộng gameplay bằng việc bổ sung chế độ Multiplayer, cho phép nhiều người chơi có thể hợp tác hoặc cạnh tranh trực tiếp, đồng thời xây dựng thêm tuyến nội dung theo chương và thiết kế các bản đồ với bối cảnh đa dạng nhằm tăng chiều sâu trải nghiệm. Bên cạnh đó, hệ thống AI và hiệu ứng sẽ được nâng cấp theo hướng nâng cao hành vi của các đối tượng địch, kết hợp bổ sung các hiệu ứng vật lý và hoạt ảnh thông qua Spine hoặc Unity Timeline để tăng mức độ chân thực và tương tác trong trò chơi.

Đối với giao diện và trải nghiệm người dùng (UI/UX), em sẽ tiến hành tối ưu hóa để đảm bảo khả năng tương thích với nhiều độ phân giải khác nhau, đồng thời tích hợp các tùy chọn cá nhân hóa như lựa chọn ngôn ngữ và thiết lập mức đồ họa nhằm mang đến trải nghiệm linh hoạt và phù hợp hơn cho người chơi.

Về chiến lược phát hành, em dự kiến đưa game lên nền tảng Google Play và App Store sau khi hoàn tất quá trình thử nghiệm mở rộng. Song song với đó, em sẽ xây dựng kế hoạch truyền thông phù hợp, kết hợp cùng hệ thống cập nhật phiên bản, bảng xếp hạng và cơ chế tiếp nhận phản hồi để duy trì sự ổn định, thu hút và phát triển cộng đồng người chơi trong thời gian dài.

3. Đánh giá cá nhân

Thông qua quá trình thực hiện đề tài, em đã củng cố và mở rộng kiến thức về lập trình và phát triển game, đồng thời làm quen với quy trình phát triển sản phẩm phần mềm hoàn chỉnh. Em hiểu rõ hơn về quản lý dự án, phân tích nhu cầu người dùng và tối ưu trải nghiệm người chơi. Bên cạnh đó, khả năng tự nghiên cứu, làm việc nhóm và tư duy sáng tạo của em cũng được nâng cao, góp phần hoàn thiện kỹ năng chuyên môn phục vụ công việc trong tương lai.

Tài liệu tham khảo

- [1] Unity Technologies, “Unity Asset Store.” Truy cập: 20/08/2025.
<https://assetstore.unity.com/>
- [2] Microsoft, “C# Language Documentation.” Truy cập: 20/08/2025.
<https://learn.microsoft.com/en-us/dotnet/csharp/>
- [3] Unity Technologies, “SRDebugger.” Truy cập: 20/08/2025.
<https://assetstore.unity.com/packages/tools/gui/srdebugger-14920>
- [4] Demigiant, “DoTween – Tween Engine for Unity.” Truy cập: 20/08/2025.
<http://dotween.demigiant.com/>
- [5] Odin Inspector, “Odin Inspector Documentation.” Truy cập: 20/08/2025.
<https://odininspector.com/>
- [6] Esoteric Software, “Spine Animation.” Truy cập: 20/08/2025.
<https://esotericsoftware.com/>
- [7] Refactoring.Guru, “Design Patterns.” Truy cập: 20/08/2025.
<https://refactoring.guru/design-patterns>
- [8] T. Lê Hoàng, “SOLID là gì? Áp dụng nguyên lý SOLID để tối ưu mã nguồn,” 2015. Truy cập: 20/08/2025.
<https://toidicodedao.com/2015/03/24/solid-la-gi-ap-dung-cac-nguyen-ly-solid-de-tro-thanh-lap-trinh-vien-code-cung/>
- [9] Unity Learn, “Finite State Machines.” Truy cập: 20/08/2025.
<https://learn.unity.com/project/finite-state-machines-1>
- [10] Unity Ads, Axon, Google AdMob – “Ads Integration Guides.” Truy cập: 20/08/2025.
<https://docs.unity.com>
<https://support.axon.ai>
<https://developers.google.com/admob/unity>
- [11] J. Schell, The Art of Game Design: A Book of Lenses, CRC Press, 2019.
- [12] Unity Learn, “Introduction to Object Pooling.” Truy cập: 20/08/2025.
<https://learn.unity.com/tutorial/introduction-to-object-pooling>

[13] R. Nystrom, Game Programming Patterns, 2014.

<http://gameprogrammingpatterns.com/>

[14] GitLab, “Version Control Tools.” Truy cập: 20/08/2025.

<https://about.gitlab.com/>

[15] Fork.dev, “Fork Git Client.” Truy cập: 20/08/2025.

<https://git-fork.com/>

[16] K. H. Nam, “KhoaLuanTotNghiep_KhuongHoaiNam – GitHub Repository,” 2025. Truy cập: 20/11/2025.

https://github.com/Khuonghoainam1/KhoaLuanTotNghiep_KhuongHoaiNam